UNIT II

Logic, Knowledge, Uncertainty & Intro to ML

- **Ch 5 · First-Order Logic**: syntax & semantics, quantifiers, using FOL, knowledge engineering
- **Ch 6 · Inference in FOL**: propositional vs FOL inference, unification & lifting, forward chaining, backward chaining, resolution
- **Ch 7 · Knowledge Graphs & Ontologies** self-study topic
- **Ch 8 · Reasoning under Uncertainty**: conditional probability, Bayes' theorem, MAP, Bayesian networks, Naive Bayes
- **Ch 9 · Introduction to Machine Learning**: definitions, terminology, key tasks, well-posed problems, types of ML, Find-S, Candidate-Elimination

Note: the revised 2026 syllabus lists First-Order Logic under Unit II; the older copy lists it at the end of Unit I. Either way it's in CIE-1.

5 First-Order Logic (FOL)

Propositional logic can only say "this whole sentence is true/false". FOL lets us talk about **objects**, their **properties** and the **relations** between them, and say things about **all** or **some** objects.

5.1 Why do we need FOL? (Representation revisited)

In propositional logic (PL) every fact is one indivisible symbol: `JohnIsKing`, `JohnIsGreedy`. PL has very limited expressive power. Try writing these in PL:

- "*Some humans are intelligent*" — PL has no way to say "some".
- "*All students who study pass*" — PL needs one sentence per student:
  $\text{Student(John)} \wedge \text{Studies(John)} \Rightarrow \text{Pass(John)}$, $\text{Student(Ravi)} \wedge \text{Studies(Ravi)} \Rightarrow \text{Pass(Ravi)}$, ... (thousands of rules!)

FOL writes it once: $\forall x\ \text{Student}(x) \wedge \text{Studies}(x) \Rightarrow \text{Pass}(x)$ — one rule for every object x.

DEFINITION

First-order logic (also called **predicate logic** / first-order predicate calculus) is an extension of propositional logic that represents knowledge in terms of objects, relations (predicates), and functions, and uses variables and quantifiers (∀, ∃) to make statements about objects.

OBJECTS

Things: people, houses, numbers, colours, wars, centuries, Richard, John...

RELATIONS

Unary (properties): red, round, prime, king.
n-ary: brother-of, bigger-than, part-of, comes-between.

FUNCTIONS

Map objects to one object: father-of, best-friend, left-leg-of, third-inning-of, end-of.

A FOL statement has two parts: the **subject** (main object) and the **predicate** (a relation that binds atoms together). E.g. "*x is an integer*" → `Integer(x)`; "*Pinky is a cat*" → `Cat(Pinky)`.

5.2 PL vs FOL ★ Very likely

Propositional Logic	First-Order Logic
Uses propositions (whole statements)	Uses predicates, objects, functions
No variables	Has variables (x, y, z...)
No quantifiers	Has quantifiers ∀ (for all), ∃ (there exists)
Limited representation	More expressive
Example: $P \Rightarrow Q$	Example: $\text{Student(x)} \Rightarrow \text{Studies(x)}$
Treats statements as indivisible	Represents objects & relationships
Easier (decidable) inference	More computationally complex inference
Ontological commitment: facts	Ontological commitment: facts, objects, relations

5.3 Syntax of FOL: the basic elements ★ Very likely

Element	Examples	Meaning
Constant symbols	John, Richard, 1, A, Mumbai	Name a specific **object**
Variables	x, y, z, a, b	Stand for any object (lowercase by convention)
Predicate symbols	Brother, King, Person, >	Name a **relation** (returns true/false)
Function symbols	LeftLeg, Father, sqrt	Name a **function** (returns an object)
Connectives	¬ ∧ ∨ ⇒ ⇔	Combine sentences (same as PL)
Equality	=	Two terms refer to the same object
Quantifiers	∀, ∃	"for all", "there exists"

Convention: constant, predicate and function symbols begin with **Uppercase**; variables are **lowercase**.

Terms

A **term** is a logical expression that refers to an object. Constants and variables are terms. A **complex term** = function symbol + parenthesised list of terms: $f(t_1, \ldots, t_n)$, e.g. $\text{LeftLeg}(\text{John})$ — "John's left leg" (an object, not true/false!).

Atomic sentences

The most basic sentence: predicate symbol + parenthesised list of terms: $\text{Predicate}(\text{term}_1, \ldots, \text{term}_n)$. "Richard and John are brothers" → $\text{Brother}(\text{Richard}, \text{John})$; also $\text{Married}(\text{Father}(\text{Richard}), \text{Mother}(\text{John}))$.

Complex sentences

Atomic sentences joined by connectives: $\text{King}(\text{Richard}) \vee \text{King}(\text{John})$, $\neg\text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John})$.

> **EXAM TIP — TERM VS SENTENCE**
>
> $\text{LeftLeg}(\text{John})$ is a **term** (an object). $\text{Brother}(\text{Richard}, \text{John})$ is an **atomic sentence** (true or false). Examiners love asking this difference.

BNF grammar of FOL (Fig 8.3 of the textbook)

```
Sentence        → AtomicSentence | ComplexSentence
AtomicSentence  → Predicate | Predicate(Term, …) | Term = Term
ComplexSentence → ( Sentence ) | ¬ Sentence
                | Sentence ∧ Sentence | Sentence ∨ Sentence
                | Sentence ⇒ Sentence | Sentence ⇔ Sentence
                | Quantifier Variable, … Sentence
Term            → Function(Term, …) | Constant | Variable
Quantifier      → ∀ | ∃
Constant        → A | X1 | John | …     Variable → a | x | s | …
Predicate       → True | False | After | Loves | Raining | …
Function        → Mother | LeftLeg | …
Operator precedence:  ¬ , = , ∧ , ∨ , ⇒ , ⇔
```

5.4 Semantics: models and interpretations

A **model** in FOL contains a set of **objects** (the *domain*) plus relations and functions among them. An **interpretation** says which object/relation/function each symbol refers to.

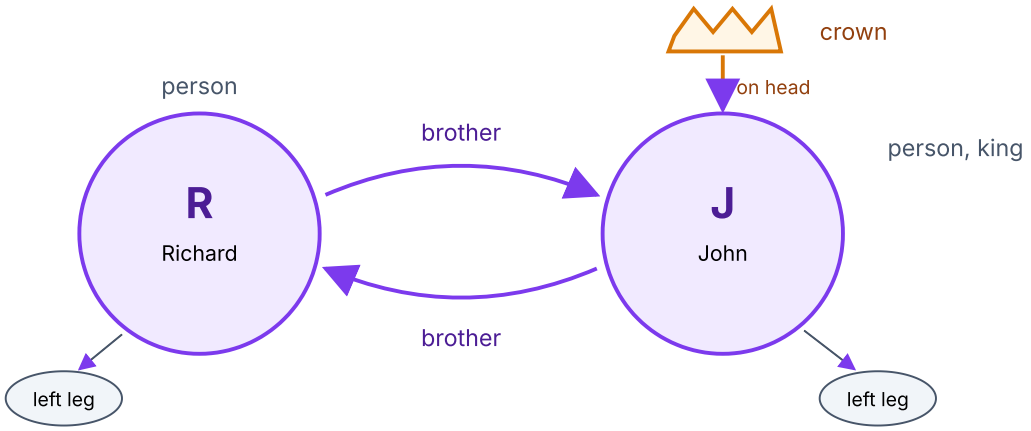


Fig 5.1 A model with 5 objects (Richard, John, their left legs, a crown), 2 binary relations (brother, on-head), 3 unary relations (person, king, crown) and 1 function (left-leg).

Interpretation used in the textbook: **Richard** → Richard the Lionheart (King of England 1189–1199); **John** → his younger brother, the evil King John (1199–1215); **Brother** → the brotherhood relation; **LeftLeg** → the "left leg" function. A relation is a set of **tuples** (objects in a fixed order, written with angle brackets), e.g. the brother relation = { ⟨Richard, John⟩, ⟨John, Richard⟩ }.

Symbols come in three kinds: **constant symbols** (objects) — Richard, John; **predicate symbols** (relations) — Brother, OnHead, Person, King, Crown; **function symbols** (functions) — LeftLeg.

5.5 Quantifiers ★ Very likely

A **quantifier** specifies the *quantity* of objects in the universe of discourse for which a statement holds — it fixes the **range and scope** of a variable.

UNIVERSAL QUANTIFIER ∀ ("FOR ALL")

True for **every** object. Read "for all x...", "everyone", "everything".
Main connective with ∀ is implication ⇒.
"All kings are persons": $\forall x\ \mathrm{King}(x) \Rightarrow \mathrm{Person}(x)$
"All men drink coffee": $\forall x\ \mathrm{Man}(x) \Rightarrow \mathrm{Drink}(x, \mathrm{Coffee})$

EXISTENTIAL QUANTIFIER ∃ ("THERE EXISTS")

True for **at least one** object. Read "there exists an x such that...", "for some x...".
Main connective with ∃ is conjunction ∧.
"King John has a crown on his head": $\exists x\ \mathrm{Crown}(x) \wedge \mathrm{OnHead}(x, \mathrm{John})$
"Some boys play cricket": $\exists x\ \mathrm{Boy}(x) \wedge \mathrm{Play}(x, \mathrm{Cricket})$

COMMON MISTAKE — #1 SOURCE OF LOST MARKS

Wrong: $\forall x\ \mathrm{King}(x) \wedge \mathrm{Person}(x)$ → says "*everything* is a king and a person" (false!).
Wrong: $\exists x\ \mathrm{Crown}(x) \Rightarrow \mathrm{OnHead}(x, \mathrm{John})$ → true if there's *anything* that's not a crown — too weak.
Rule: ∀ goes with ⇒, ∃ goes with ∧.

MEMORY TRICK

"**A**ll → **A**rrow (⇒)", "**E**xists → **E**and (∧)". The upside-down **A** = **A**ll, backwards **E** = **E**xists.

Nested quantifiers

- "Brothers are siblings": $\forall x\ \forall y\ \mathrm{Brother}(x, y) \Rightarrow \mathrm{Sibling}(x, y)$
- "Everybody loves somebody": $\forall x\ \exists y\ \mathrm{Loves}(x, y)$ vs "There is someone loved by everyone": $\exists y\ \forall x\ \mathrm{Loves}(x, y)$ — order matters!
- ∃! (or $\exists_1$) means "there exists exactly one".

Connection between ∀ and ∃ (De Morgan for quantifiers)

$$\neg\forall x\ P(x) \equiv \exists x\ \neg P(x) \qquad \neg\exists x\ P(x) \equiv \forall x\ \neg P(x) \qquad \forall x\ P(x) \equiv \neg\exists x\ \neg P(x) \qquad \exists x\ P(x) \equiv \neg\forall x\ \neg P(x)$$

"Not everyone likes ice-cream" = "someone doesn't like ice-cream".

Equality

The equality symbol = says two terms refer to the **same object**: $\text{Father}(\text{John}) = \text{Henry}$. With negation it insists two terms are different: "Richard has at least two brothers": $\exists x\ \exists y\ \text{Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg(x = y)$.

5.6 English → FOL: practice set from class ★ Very likely

English	FOL
Marcus was a man	$\text{Man}(\text{Marcus})$
x is an integer	$\text{Integer}(x)$
Caesar was a ruler	$\text{Ruler}(\text{Caesar})$
Ravi and Rama are brothers	$\text{Brothers}(\text{Ravi}, \text{Rama})$
Marcus tried to assassinate Caesar	$\text{TryToAssassinate}(\text{Marcus}, \text{Caesar})$
All men drink coffee	$\forall x\ \text{Man}(x) \Rightarrow \text{Drink}(x, \text{Coffee})$
Every man respects his parent	$\forall x\ \text{Man}(x) \Rightarrow \text{Respects}(x, \text{Parent})$
Some boys play cricket	$\exists x\ \text{Boy}(x) \wedge \text{Play}(x, \text{Cricket})$
Not all students like both Maths and Science	$\neg\forall x\ [\text{Student}(x) \Rightarrow \text{Like}(x, \text{Maths}) \wedge \text{Like}(x, \text{Science})]$
Everyone is loyal to someone	$\forall x\ \exists y\ \text{Loyal}(x, y)$
Everyone loves everyone	$\forall x\ \forall y\ \text{Loves}(x, y)$
All boys like cricket	$\forall x\ \text{Boy}(x) \Rightarrow \text{Like}(x, \text{Cricket})$
Some boys like cricket	$\exists x\ \text{Boy}(x) \wedge \text{Like}(x, \text{Cricket})$
Every person likes ice-cream	$\forall x\ \text{Person}(x) \Rightarrow \text{Likes}(x, \text{IceCream})$
All kings who are greedy are evil	$\forall x\ \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
No student failed	$\neg\exists x\ \text{Student}(x) \wedge \text{Failed}(x) \equiv \forall x\ \text{Student}(x) \Rightarrow \neg\text{Failed}(x)$

NOTE ON A SLIDE

The class slide writes "Everyone loves everyone" as $\forall x\ \exists y\ \text{loves}(x, y)$ — that formula actually means "everyone loves *someone*". The correct form is $\forall x\ \forall y\ \text{Loves}(x, y)$. Also "Every man respects his parent" is more precisely $\forall x\ \forall y\ \text{Man}(x) \wedge \text{Parent}(y, x) \Rightarrow \text{Respects}(x, y)$; write the precise form if you have time.

EXAM TIP — 4-STEP RECIPE FOR TRANSLATING

1. Find the **objects** (constants) and the **properties/relations** (predicates).
2. Look for quantity words: *all, every, each, any* → ∀; *some, a, there is, at least one* → ∃.
3. Use ⇒ under ∀, ∧ under ∃.
4. "Not all..." → put ¬ in front of ∀; "No one.../None..." → ¬∃ (or ∀ ... ⇒ ¬...).

5.7 Using FOL: assertions, queries and domains

Sentences are added to a KB with **TELL** (exactly as in PL) — these are **assertions**:

```
TELL(KB, King(John))        TELL(KB, Person(Richard))        TELL(KB, ∀x King(x) ⇒ Person(x))
```

Questions are asked with **ASK** — these are **queries** or **goals**: `ASK(KB, King(John))` → true. `ASK(KB, ∃x Person(x))` → true; **ASKVARS**(KB, Person(x)) returns a **substitution** (binding list) like {x/John} and {x/Richard}.

A **domain** is some part of the world about which we wish to express knowledge — e.g. the **kinship domain** and the **Wumpus world**.

The kinship (family) domain

- **Objects:** people
- **Unary predicates:** Male, Female
- **Binary predicates:** Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt, Uncle
- **Functions:** Mother, Father
- Facts: "Elizabeth is the mother of Charles", "Charles is the father of William". Rule: "One's grandmother is the mother of one's parent".

Brothers are siblings: ∀x,y Brother(x,y) ⇒ Sibling(x,y)
Male and female are disjoint: ∀x Male(x) ⇔ ¬Female(x)
Parent/child are inverse: ∀p,c Parent(p,c) ⇔ Child(c,p)
Mother definition: ∀m,c Mother(c)=m ⇔ Female(m) ∧ Parent(m,c)
Husband: ∀w,h Husband(h,w) ⇔ Male(h) ∧ Spouse(h,w)
Grandparent: ∀g,c Grandparent(g,c) ⇔ ∃p Parent(g,p) ∧ Parent(p,c)
Sibling: ∀x,y Sibling(x,y) ⇔ x≠y ∧ ∃p Parent(p,x) ∧ Parent(p,y)

The Wumpus world in FOL

- **Percept** = predicate with the percept list and time: Percept([Stench, Breeze, Glitter, None, None], 5)
- **Actions** are logical terms: Turn(Right), Turn(Left), Forward, Shoot, Grab, Release, Climb. **Query:** ∃a BestAction(a, 5) → answer {a/Grab}.
- **Perception** (percept ⇒ fact): ∀t,s,b,m,c Percept([s, b, Glitter, m, c], t) ⇒ Glitter(t)
- **Reflex behaviour:** ∀t Glitter(t) ⇒ BestAction(Grab, t)
- **Environment:** objects = squares, pits, wumpus. Adjacency: ∀x,y,a,b Adjacent([x,y],[a,b]) ⇔ [a,b] ∈ {[x+1,y], [x−1,y],[x,y+1],[x,y−1]}; unary predicate Pit(x).
- Agent infers square properties from its percept: ∀s,t At(Agent, s, t) ∧ Breeze(t) ⇒ Breezy(s) (Breezy has no time argument). Having found which squares are breezy/smelly, the agent can deduce where the pits and the wumpus are: ∀s Breezy(s) ⇔ ∃r Adjacent(r, s) ∧ Pit(r).

5.8 Knowledge engineering in FOL ★ Very likely

DEFINITION

Knowledge engineering is the general process of constructing a knowledge base. A **knowledge engineer** investigates a particular domain, learns what concepts are important in it, and creates a formal representation of the objects and relations in the domain.

The 7 steps of the knowledge-engineering process

1. **Identify the task** — figure out the range of questions the KB must answer.
2. **Assemble the relevant knowledge** — knowledge acquisition / requirement analysis (work with domain experts).
3. **Decide on a vocabulary** of predicates, functions and constants — this vocabulary is the domain's **ontology**.
4. **Encode general knowledge about the domain** — write axioms (rules) that hold for all instances.
5. **Encode a description of the specific problem instance** — simple atomic sentences about this case.
6. **Pose queries to the inference procedure and get answers.**
7. **Debug the knowledge base** — fix missing or wrong axioms.

MEMORY TRICK

"**I** **A**m **D**oing **G**reat **S**o **Q**uit **D**oubting" → **I**dentify, **A**ssemble, **D**ecide vocabulary, **G**eneral knowledge, **S**pecific instance, **Q**uery, **D**ebug.

Worked example: the one-bit full adder (electronic circuits domain)

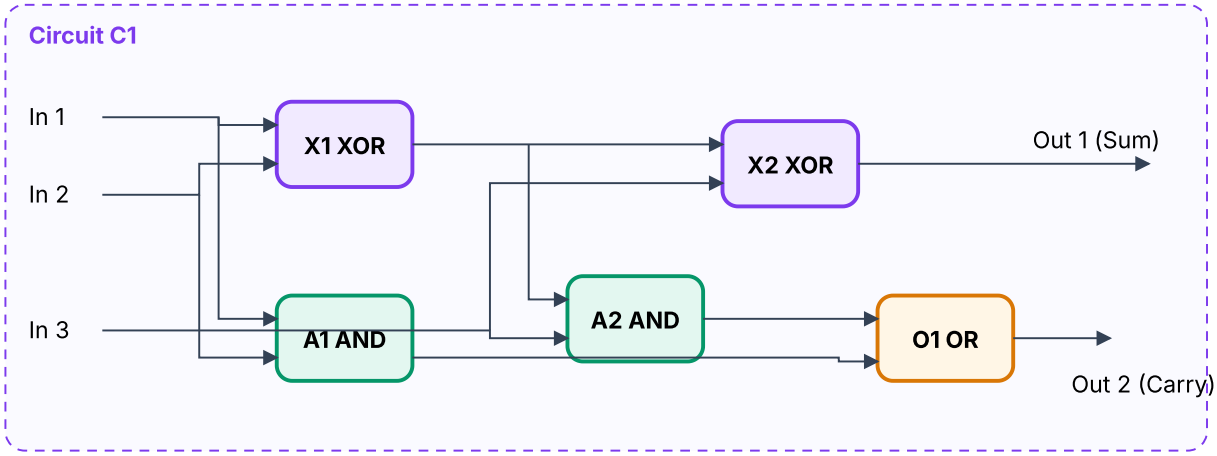


Fig 5.2 One-bit full adder C1: two XOR gates (X1, X2), two AND gates (A1, A2), one OR gate (O1). Out1 = sum bit, Out2 = carry bit.

1. **Identify the task.** Level 1 (functionality): *Does the circuit add properly? What is the output of gate A2 if all inputs are high?* Level 2 (structure): *Which gate is connected to the first input terminal? Does the circuit have feedback loops?*
2. **Assemble knowledge.** Circuits are made of **wires** and **gates**. Signals flow along wires to gate input terminals; each gate produces an output that flows on. Four gate types: AND, OR, XOR, NOT. All have two inputs and one output, except NOT (one input). Delays, power, cost are irrelevant here.
3. **Decide vocabulary.** Gates are objects named by constants (X1, X2, A1, A2, O1) — $\mathrm{Gate}(\mathrm{X1})$; gate type via function: $\mathrm{Type}(\mathrm{X1}) = \mathrm{XOR}$; circuit predicate $\mathrm{Circuit}(\mathrm{C1})$; terminals $\mathrm{Terminal}(x)$; input/output terminals by functions $\mathrm{In}(1, \mathrm{X1})$, $\mathrm{Out}(1, \mathrm{X1})$; connectivity $\mathrm{Connected}(\mathrm{Out}(1, \mathrm{X1}), \mathrm{In}(1, \mathrm{X2}))$; signal values 1 and 0 and function $\mathrm{Signal}(t)$.
4. **Encode general knowledge** (domain axioms):

If two terminals are connected, they have the same signal: $\forall t_1,t_2$ Terminal(t_1) ∧ Terminal(t_2) ∧ Connected(t_1,t_2) ⇒ Signal(t_1) = Signal(t_2)
Signal is either 1 or 0: $\forall t$ Terminal(t) ⇒ Signal(t) = 1 ∨ Signal(t) = 0 ; and 1 ≠ 0
Connected is commutative: $\forall t_1,t_2$ Connected(t_1,t_2) ⇔ Connected(t_2,t_1)
AND gate: ∀g Gate(g) ∧ Type(g)=AND ⇒ [Signal(Out(1,g)) = 0 ⇔ ∃n Signal(In(n,g)) = 0]
OR gate: ∀g Gate(g) ∧ Type(g)=OR ⇒ [Signal(Out(1,g)) = 1 ⇔ ∃n Signal(In(n,g)) = 1]
XOR gate: ∀g Gate(g) ∧ Type(g)=XOR ⇒ [Signal(Out(1,g)) = 1 ⇔ Signal(In(1,g)) ≠ Signal(In(2,g))]
NOT gate: ∀g Gate(g) ∧ Type(g)=NOT ⇒ Signal(Out(1,g)) ≠ Signal(In(1,g))

5. **Encode the specific instance:** Circuit(C1) ∧ Arity(C1,3,2); Gate(X1) ∧ Type(X1)=XOR, Gate(X2) ∧ Type(X2)=XOR, Gate(A1) ∧ Type(A1)=AND, Gate(A2) ∧ Type(A2)=AND, Gate(O1) ∧ Type(O1)=OR; connections such as Connected(Out(1,X1), In(1,X2)), Connected(In(1,C1), In(1,X1)), Connected(Out(1,X1), In(2,A2)), Connected(Out(1,A2), In(1,O1)), Connected(Out(1,A1), In(2,O1)), Connected(Out(1,X2), Out(1,C1)), Connected(Out(1,O1), Out(2,C1)), ...
6. **Pose queries.** "What combination of inputs makes the first output of C1 = 0 and the second output = 1?"
$\exists i_1,i_2,i_3$ Signal(In(1,C1))=i_1 ∧ Signal(In(2,C1))=i_2 ∧ Signal(In(3,C1))=i_3 ∧ Signal(Out(1,C1))=0 ∧ Signal(Out(2,C1))=1
Answers: (i_1,i_2,i_3) = (1,1,0), (1,0,1), (0,1,1) — i.e. exactly two inputs are 1 (sum 0, carry 1). Another query: find all possible sets of values of all terminals → the full adder truth table.
7. **Debug the KB.** E.g. if we forgot to assert 1 ≠ 0, some queries return nothing; check outputs of every gate against expectations.

LIKELY EXAM QUESTIONS — CH 5

1. Differentiate propositional logic and first-order logic. (5M)
2. Explain the syntax of FOL with its basic elements; define term, atomic sentence and complex sentence. (6M)
3. Explain universal and existential quantifiers with examples. Why is ⇒ used with ∀ and ∧ with ∃? (5M)
4. Translate the given English sentences into FOL. (5–6M — almost always asked)
5. What is knowledge engineering? Explain the steps with the electronic-circuit (full adder) example. (10M)
6. Write FOL axioms for the kinship domain / Wumpus world. (5M)

60-SECOND RECAP

FOL = objects + relations + functions + variables + quantifiers. Term → object; atomic sentence → true/false. ∀ with ⇒, ∃ with ∧; ¬∀ = ∃¬. TELL adds assertions, ASK poses queries. Knowledge engineering: Identify task → Assemble knowledge → Decide vocabulary → General axioms → Specific instance → Query → Debug.

6 Inference in First-Order Logic

Inference = deriving a *new* sentence from sentences we already know. Example: from $\text{Human}(\text{Socrates})$ and $\forall x\ \text{Human}(x) \Rightarrow \text{Mortal}(x)$ we derive $\text{Mortal}(\text{Socrates})$.

Method	Basic idea (one line to remember)	Starts with	Direction
Propositional inference	Convert FOL sentences into specific (ground) propositions, then use PL inference	Ground propositions	Depends on method
Unification	Find substitutions that make two expressions identical	Two FOL expressions	Matching
Lifting	Perform inference directly with variables (Generalized Modus Ponens)	FOL rules + facts	Generalized
Forward chaining	Start from facts and derive new facts	Known facts	Forward (facts → goal)
Backward chaining	Start from goal and find supporting facts	Query / goal	Backward (goal → facts)
Resolution	Prove a statement by contradiction	KB + negated query	Refutation

6.1 Propositional vs first-order inference

Propositional logic works with simple statements that are either true or false (`JohnIsKing`, `JohnIsGreedy`, `JohnIsEvil`). FOL works with objects, properties, relationships and variables ($\text{King}(\text{John})$, $\forall x\ \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$). One way to do FOL inference is to remove the quantifiers (instantiate the variables) and then use propositional inference.

Substitution

DEFINITION

A **substitution** θ is a finite set $\{v_1/t_1, \ldots, v_n/t_n\}$ where each v_i is a **variable**, each t_i is a **term** different from v_i, and no variable appears twice. $\text{SUBST}(\theta, \alpha)$ = the sentence obtained by applying θ to α.

$$\alpha = \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x), \quad \theta = \{x/\text{John}\}$$
$$\text{SUBST}(\theta, \alpha) = \text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$$

Notation: the textbook writes {x/John} ("x is replaced by John"). Some class slides write it the other way round, [John/x] or {t/v}. Both mean the same — just be consistent in your answer.

Inference rules for quantifiers ★ Very likely

Rule	Form	Meaning & example
Universal Instantiation (UI) a.k.a. universal elimination	$\frac{\forall v\ \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$ g = any ground term	From "for all" infer it for any specific object. Can be applied many times. "Every person likes ice-cream" ⊢ "John likes ice-cream". From $\forall x\ \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ infer $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$, $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$, $\text{King}(\text{Father}(\text{John})) \wedge \ldots$
Existential Instantiation (EI) a.k.a. existential elimination	$\frac{\exists v\ \alpha}{\text{SUBST}(\{v/k\}, \alpha)}$ k = NEW constant	Give the "something" a brand-new name. Applied only once; the constant must not appear anywhere else in the KB. $\exists x\ \text{Crown}(x) \wedge \text{OnHead}(x, \text{John}) \vdash \text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$. The new constant is a **Skolem constant**; EI is a special case of **Skolemization**. New KB is not logically equivalent to the old one, but is **satisfiable iff** the old one was (inferentially equivalent).
Universal Generalization (UG)	$\frac{P(c) \text{ for arbitrary } c}{\forall x\ P(x)}$	If P(c) holds for an **arbitrary** element c, conclude ∀x P(x). "A byte contains 8 bits" (any byte) ⊢ "All bytes contain 8 bits".
Existential Introduction / Generalization (EG)	$\frac{P(c)}{\exists x\ P(x)}$	If some specific c has property P, then something has P. "Pinky got good marks in English" ⊢ "Someone got good marks in English".

MEMORY TRICK

UI = **U**se any constant, as many times as you like. **E**I = **E**xactly once, with a fr**e**sh name.

Propositionalization (reduction to propositional inference)

Apply UI with **every** ground term and EI once per ∃ → the KB becomes propositional (treat $\text{King}(\text{John})$ as one symbol like `JohnIsKing`) → use any PL algorithm. **Problems:**

- With function symbols there are infinitely many ground terms: Father(John), Father(Father(John)), … (Herbrand's theorem: if α is entailed, a proof exists using a finite subset — so the method is complete but only **semidecidable**: it may never halt if α is not entailed).
- It generates lots of irrelevant sentences (e.g. $\text{Greedy}(\text{Richard})$ …) — very inefficient. This motivates **lifting**.

6.2 Unification ★ Very likely

DEFINITION

Unification is the process of finding a substitution θ that makes two different logical expressions look identical:

$$\text{UNIFY}(p, q) = \theta \quad \text{where} \quad \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

θ is called the **unifier**. Unification is the key component of all first-order inference algorithms (forward chaining, backward chaining, resolution).

Conditions for unification

1. **Predicate symbols must be the same** — atoms with different predicate symbols can never be unified (Knows(...) vs Loves(...) → FAIL).
2. **Number of arguments must be identical** (Knows(A, B) vs Knows(A) → FAIL).
3. A variable can't take two different values at once, and a variable can't be unified with a term containing itself (**occurs check**: x vs F(x) → FAIL).

Worked examples (textbook + class)

p	q	θ = UNIFY(p, q)
Knows(John, x)	Knows(John, Jane)	{x/Jane}
Knows(John, x)	Knows(y, Bill)	{x/Bill, y/John}
Knows(John, x)	Knows(y, Mother(y))	{y/John, x/Mother(John)}
Knows(John, x)	Knows(x, Elizabeth)	**FAIL** (x can't be John and Elizabeth)
p(x, F(y))	p(a, F(g(z)))	{x/a, y/g(z)} (slide: [a/x, g(z)/y])
Q(a, g(x, a), f(y))	Q(a, g(f(b), a), x)	{x/f(b), y/b} → both become Q(a, g(f(b), a), f(b))
Student(x)	Student(Anu)	{x/Anu}

WORKED EXAMPLE — STEP BY STEP

Unify Q(a, g(x, a), f(y)) and Q(a, g(f(b), a), x):

1. Same predicate Q, 3 arguments each ✔.
2. Arg 1: a = a ✔ (no substitution).
3. Arg 2: g(x, a) vs g(f(b), a) → x = f(b). Substitution so far {x/f(b)}. Apply it to the rest: arg 3 of q becomes f(b).
4. Arg 3: f(y) vs f(b) → y = b. Final θ = **{x/f(b), y/b}**.

Standardizing apart

The failure of Knows(John, x) vs Knows(x, Elizabeth) is only because both sentences happen to use the name x. Variables in different sentences are independent, so we rename the variables of one sentence before unifying: Knows(x_{17}, Elizabeth) → θ = {x/Elizabeth, x_{17}/John}. Essential for backward chaining and resolution.

Most General Unifier (MGU)

There can be many unifiers. UNIFY(Knows(John, x), Knows(y, z)) could return {y/John, x/z} or {y/John, x/John, z/John}. The first is **more general** (places fewer restrictions). For every unifiable pair there is a single MGU, unique up to renaming — algorithms always return it.

The unification algorithm (class version)

```
UNIFY(L1, L2)
Step 1: If L1 or L2 is a variable or constant:
    a) If L1 and L2 are identical → return NIL (empty substitution)
    b) Else if L1 is a variable: if L1 occurs in L2 → FAIL, else return {L1/L2}
    c) Else if L2 is a variable: if L2 occurs in L1 → FAIL, else return {L2/L1}
    d) Else → FAIL
Step 2: If the initial predicate symbols of L1 and L2 differ → FAIL
Step 3: If L1 and L2 have a different number of arguments → FAIL
Step 4: SUBST ← NIL
Step 5: For i ← 1 to number of arguments in L1:
    a) S ← UNIFY(i-th argument of L1, i-th argument of L2)
    b) If S = FAIL → return FAIL
    c) If S ≠ NIL → apply S to the remainders of L1 and L2;  SUBST ← APPEND(S, SUBST)
Step 6: Return SUBST
```

Textbook version: `UNIFY(x, y, θ)` — if θ = failure return failure; if x = y return θ; if x is a variable return UNIFY-VAR(x, y, θ); if y is a variable return UNIFY-VAR(y, x, θ); if both compound, unify their operators then their argument lists; if both lists, unify FIRST then REST; else failure. `UNIFY-VAR` does the occurs check.

6.3 Lifting & Generalized Modus Ponens ★ Very likely

DEFINITION

Lifting is applying inference rules directly to FOL sentences containing variables (using unification), instead of first converting everything into propositional logic. Lifting was introduced by **Alan Robinson** — it was the key innovation of his resolution procedure. The lifted version of Modus Ponens is **Generalized Modus Ponens (GMP)**.

$$\frac{p_1', p_2', \ldots, p_n', \quad (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{SUBST(\theta, q)} \quad \text{where } SUBST(\theta, p_i') = SUBST(\theta, p_i) \text{ for all } i$$

n atomic sentences p_i' + one implication → if one θ makes every p_i' match p_i, conclude q with θ applied.

EXAMPLE 1 (TEXTBOOK)

p_1' = King(John), p_2' = Greedy(y)
rule: King(x) ∧ Greedy(x) ⇒ Evil(x)
θ = {x/John, y/John}
SUBST(θ, q) = **Evil(John)**

EXAMPLE 2 (CLASS)

facts: Student(John), Studies(John)
rule: Student(x) ∧ Studies(x) ⇒ Pass(x)
θ = {x/John}
Pass(x) becomes **Pass(John)**

WHY LIFTING MATTERS

Without lifting we'd propositionalize: Student(John) ⇒ Pass(John), Student(Ravi) ⇒ Pass(Ravi), Student(Sita) ⇒ Pass(Sita)... With lifting we keep $\text{Student}(x) \Rightarrow \text{Pass}(x)$ and use {x/John} **only when needed**. So lifting avoids unnecessary grounding of all possible objects. GMP is sound.

6.4 Forward chaining ★ Very likely

The **inference engine** is the part of a system that applies logical rules to the KB to deduce new information. It works in two modes: **forward chaining** (forward reasoning) and **backward chaining**

(backward reasoning).

DEFINITION

Forward chaining starts from the known facts, triggers all rules whose premises are satisfied, and adds their conclusions to the known facts. The process repeats until the query is answered or no new facts are added (a *fixed point*). It is **data-driven**: Facts → Rules → New facts → Rules → ...

It works on **first-order definite clauses**: disjunctions of literals with **exactly one positive literal**, i.e. either an atomic fact (King(John)) or an implication whose premise is a conjunction of positive literals (King(x) ∧ Greedy(x) ⇒ Evil(x)).

WARM-UP EXAMPLE

Facts: Student(John), Studies(John). Rule 1: Student(x) ∧ Studies(x) ⇒ Pass(x). Rule 2: Pass(x) ⇒ Graduate(x). Query: Graduate(John)?
Step 1: Rule 1 matches with {x/John} → add **Pass(John)**. **Step 2:** Rule 2 matches → add **Graduate(John)** ✔.

The classic crime example ("Colonel West") ★ Very likely

PROBLEM

"The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American." Prove that West is a criminal.

Step 1 — convert every sentence into a first-order definite clause (x, y, z are variables):

#	English	Definite clause
1	It is a crime for an American to sell weapons to hostile nations	$\text{American}(x) \land \text{Weapon}(y) \land \text{Sells}(x, y, z) \land \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
2	Nono has some missiles: ∃x Owns(Nono, x) ∧ Missile(x) — use EI with new constant M_1	$\text{Owns}(\text{Nono}, M_1)$
3	(same sentence, second half)	$\text{Missile}(M_1)$
4	All of its missiles were sold to it by Colonel West	$\text{Missile}(x) \land \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
5	Missiles are weapons (background knowledge)	$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$
6	An enemy of America counts as hostile (background)	$\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
7	West, who is American	$\text{American}(\text{West})$
8	Nono, an enemy of America	$\text{Enemy}(\text{Nono}, \text{America})$

Step 2 — iterate. Iteration 0 (facts with no implications): American(West), Missile(M_1), Owns(Nono, M_1), Enemy(Nono, America).
Iteration 1: rule 4 {x/M_1} → **Sells(West, M_1, Nono)**; rule 5 {x/M_1} → **Weapon(M_1)**; rule 6 {x/Nono} → **Hostile(Nono)**.
Iteration 2: rule 1 {x/West, y/M_1, z/Nono} → **Criminal(West)** ✔ — goal reached; no new facts possible after this (fixed point).

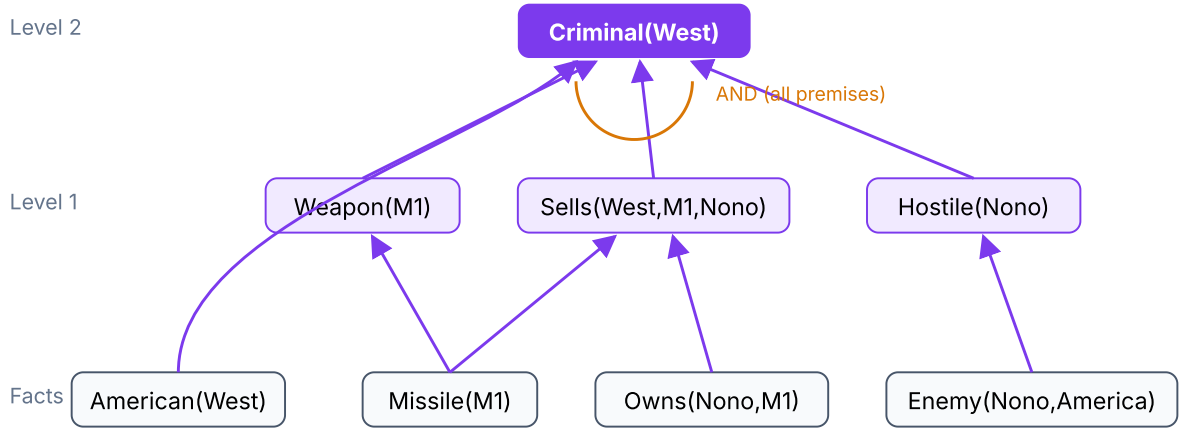


Fig 6.1 Forward-chaining proof tree: facts at the bottom, facts inferred in iteration 1 in the middle, Criminal(West) inferred in iteration 2 at the top.

EXAM TIP — HOW TO WRITE THIS 10-MARK ANSWER

(1) Write the 8 definite clauses in a table. (2) Show iteration-by-iteration which rule fires with which substitution. (3) Draw the proof tree (bottom-up). (4) Conclude "Hence Criminal(West) is proved". Mention **EI** for "Nono has some missiles" — examiners look for that.

Second class example: "John likes peanuts" (forward chaining)

Sentences: (1) John likes all kinds of food. (2) Apples are food. (3) Chicken is food. (4) Anything anyone eats and isn't killed by is food. (5) Bill eats peanuts and is still alive. (6) Sue eats everything Bill eats.

(i) Translation into predicate logic

1. ∀x Food(x) ⇒ Likes(John, x)
2. Food(Apple)
3. Food(Chicken)
4. ∀x ∀y Eats(x, y) ∧ ¬Killed(x) ⇒ Food(y)
5. Eats(Bill, Peanut) ∧ ¬Killed(Bill)
6. ∀x Eats(Bill, x) ⇒ Eats(Sue, x)

(ii) Forward chaining — working memory (WM): Food(Apple), Food(Chicken), Eats(Bill, Peanut), ¬Killed(Bill)

- Rule 6, {x/Peanut}: add **Eats(Sue, Peanut)**
- Rule 4, {x/Bill, y/Peanut}: add **Food(Peanut)**
- Rule 1, {x/Peanut}: add **Likes(John, Peanut)** ✓

Hence proved by forward chaining.

Properties, advantages and disadvantages of forward chaining

- **Sound and complete** for first-order definite clauses. **Datalog** = first-order definite clauses with **no function symbols** (e.g. the crime KB) — FC always terminates on Datalog in polynomial time.
- With function symbols FC may run forever if the query is not entailed (semidecidable).
- **Advantages:** data-driven (starts from available facts); can derive many conclusions until a fixed point; widely used in **deductive databases** and **production systems** (expert systems, e.g. rule engines).
- **Disadvantage:** with 1000 rules and the question "Is John a graduate?", FC may derive many facts irrelevant to the goal — the main source of inefficiency → motivates backward chaining.

FOL-FC-ASK algorithm (idea): repeat { new ← {}; for each rule, standardize it apart; for each θ such that SUBST(θ, $p_1 \wedge \ldots \wedge p_n$) = SUBST(θ, $p_1' \wedge \ldots \wedge p_n'$) for facts p′ in KB: q′ ← SUBST(θ, q); if q′ is not a renaming of a known sentence, add to new; if q′ unifies with the query, return the unifier } until new is empty; add new to KB.

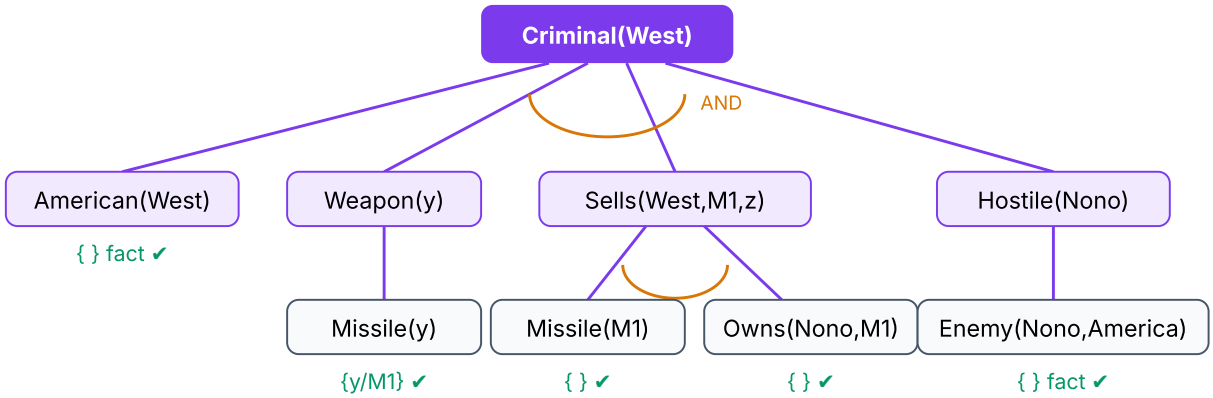
6.5 Backward chaining ★ Very likely

DEFINITION

Backward chaining starts from the **goal (query)** and works backwards: find rules whose conclusion unifies with the goal, and turn their premises into **new sub-goals**, recursively, until all sub-goals are known facts. It is **goal-driven / goal-directed reasoning**: Goal → Rules → Sub-goals → Facts. Backward chaining is a depth-first search algorithm.

WARM-UP EXAMPLE

Rule: Student(x) ∧ Studies(x) ⇒ Pass(x). Facts: Student(John), Studies(John). Query: Pass(John)? **Step 1** Start with goal Pass(John); ask "which rule can conclude Pass(x)?" → the rule above. **Step 2** Unify Pass(x) with Pass(John): x = John. **Step 3** New sub-goals: Student(John) AND Studies(John). **Step 4** Both are facts → Pass(John) is proved ✔.



Read top-down, left-to-right (depth-first). Substitutions found: {x/West, y/M1, z/Nono}

Fig 6.2 Backward-chaining proof tree for Criminal(West). Each sub-goal is proved left to right, depth first; the curved arc marks an AND node.

"John likes peanuts" by backward chaining

```
Goal: Likes(John, Peanut)
  └─ use rule 1 {x/Peanut}:  Food(Peanut)
       └─ use rule 4 {y/Peanut}:  Eats(x, Peanut) ∧ ¬Killed(x)
            ├─ Eats(x, Peanut)  {x/Bill}  → Eats(Bill, Peanut)   fact 5 ✔
            └─ ¬Killed(x)       {x/Bill}  → ¬Killed(Bill)        fact 5 ✔
  ⇒ Food(Peanut) ✔  ⇒  Likes(John, Peanut) ✔
```

AND/OR interpretation

Backward chaining is an **AND/OR search**. **OR**: several rules may prove the same goal — e.g. Pass(John) via $\text{Student}(x) \wedge \text{Studies}(x) \Rightarrow \text{Pass}(x)$ or $\text{TopRank}(x) \Rightarrow \text{Pass}(x)$; any one succeeding is enough. **AND**: once a rule is chosen, all its premises (Student(John) AND Studies(John)) must be proved.

ADVANTAGES

- Goal-directed; avoids many irrelevant facts
- Good for answering specific queries
- Foundation of **logic programming**; **Prolog** uses depth-first backward chaining

DISADVANTAGES

- Repeated states & redundant computations
- Can fall into **infinite loops** (with unsuitable rule ordering)
- Incomplete as plain DFS

6.6 Forward vs backward chaining ★ Very likely

Feature	Forward chaining	Backward chaining
Also called	Data-driven, bottom-up	Goal-driven, top-down
Starting point	Known facts	Goal / query
Direction	Facts → conclusion	Goal → premises (sub-goals)
Main question	"What can I derive?"	"What do I need to prove?"
Irrelevant inference	More likely	Less likely
Search style	Breadth-first-like, data-driven	Depth-first, AND/OR search
Useful for	Databases, production/expert systems, monitoring	Query answering, diagnosis, logic programming
Used by Prolog?	No	Yes
Infinite loops	Generally not (finite Datalog closure)	Yes, with unsuitable rule ordering
Example	Crime KB iterations up to Criminal(West)	Criminal(West) tree down to facts

6.7 Resolution ★ Very likely

DEFINITION

Resolution is a theorem-proving technique that builds **refutation proofs** (proofs by contradiction). It was invented by mathematician **John Alan Robinson in 1965**. To prove $KB \models \alpha$, we show that $KB \wedge \neg\alpha$ is **unsatisfiable** by deriving the **empty clause** $\square$ (a contradiction).

$$KB \models \alpha \iff KB \wedge \neg\alpha \text{ is unsatisfiable} \iff KB \wedge \neg\alpha \vdash \square$$

The resolution rule

Propositional:

$$\frac{P \vee Q, \quad \neg Q}{P} \qquad \frac{\ell_1 \vee \ldots \vee \ell_k, \quad m_1 \vee \ldots \vee m_n}{\text{remaining literals}}$$

ℓ_i and m_j are complementary (Q and $\neg Q$): remove both, OR the rest.

First-order: complementary literals need not look identical — they must **unify**: if $\text{UNIFY}(\ell_i, \neg m_j) = \theta$, the resolvent is $\text{SUBST}(\theta,$ rest of both clauses$)$

Terminology on the class slides: the pair of clauses being resolved are the **resolvents**' parents (slide: "resolvents"); the resulting clause is the **resolvent** (slide: "resolute"). Choosing the right pair to resolve is a matter of search.

SIMPLE FOL RESOLUTION STEP (TEXTBOOK)

C1: $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$ C2: $\neg\text{Loves}(u, v) \vee \neg\text{Kills}(u, v)$

Step 1 unify $\text{Loves}(G(x), x)$ with $\text{Loves}(u, v)$: $\theta = \{u/G(x), v/x\}$. Step 2 remove the complementary pair. Step 3 apply θ → resolvent **Animal(F(x)) ∨ ¬Kills(G(x), x)**.

Converting FOL to CNF (clause form) ★ Very likely

Resolution needs every sentence in **Conjunctive Normal Form**: a conjunction (∧) of clauses, where each clause is a disjunction (∨) of literals, e.g. $(A \vee B) \wedge (\neg C \vee D)$.

#	Step	What to do	Example
1	**Eliminate ⇔ and ⇒**	P ⇔ Q → (P ⇒ Q) ∧ (Q ⇒ P); P ⇒ Q → ¬P ∨ Q	P ⇒ Q ≡ ¬P ∨ Q
2	**Move ¬ inwards**	¬¬P → P; De Morgan; ¬∀x P → ∃x ¬P; ¬∃x P → ∀x ¬P	¬(P ∧ Q) ≡ ¬P ∨ ¬Q
3	**Standardize variables**	Each quantifier gets its own unique variable name	∀x Man(x), ∀x Happy(x) → ∀x Man(x), ∀y Happy(y)
4	Move quantifiers left (prenex form)	Bring all quantifiers to the front	P(x) ∧ ∀y Q(y) → ∀y [P(x) ∧ Q(y)]
5	**Skolemize** (remove ∃)	Replace ∃ variable by a **Skolem constant**, or a **Skolem function** of the enclosing ∀ variables	∃x Smile(x) → Smile(A); ∀y ∃x Loves(x, y) → ∀y Loves(F(y), y)
6	**Drop ∀**	All remaining variables are implicitly universal	∀x Smile(x) → Smile(x)
7	**Distribute ∨ over ∧**	(P ∧ Q) ∨ R → (P ∨ R) ∧ (Q ∨ R); (P ∨ Q) ∨ R → P ∨ Q ∨ R	(P ∧ Q) ∨ R → (P ∨ R) ∧ (Q ∨ R)
8	**Split conjuncts** into separate clauses	Each conjunct becomes one clause	(P ∨ R) ∧ (Q ∨ R) → C1: P ∨ R, C2: Q ∨ R
9	**Standardize apart** again	No two clauses share a variable name	P(x) ∨ Q(x), ¬P(x) ∨ R(x) → … ¬P(y) ∨ R(y)

SKOLEM CONSTANT VS SKOLEM FUNCTION

If an ∃ sits **inside** a ∀, the "something" may depend on the ∀ variable, so use a **function**: "Everyone has a heart" ∀x ∃y Heart(y, x) → Heart(H(x), x), not Heart(H, x) (which would mean everyone shares one heart!).

FULL CNF CONVERSION (TEXTBOOK): "EVERYONE WHO LOVES ALL ANIMALS IS LOVED BY SOMEONE"

∀x [∀y Animal(y) ⇒ Loves(x, y)] ⇒ [∃y Loves(y, x)]

1. ⇒ elim: ∀x ¬[∀y ¬Animal(y) ∨ Loves(x, y)] ∨ [∃y Loves(y, x)]
2. ¬ in: ∀x [∃y ¬(¬Animal(y) ∨ Loves(x, y))] ∨ [∃y Loves(y, x)] → ∀x [∃y Animal(y) ∧ ¬Loves(x, y)] ∨ [∃y Loves(y, x)]
3. standardize: ∀x [∃y Animal(y) ∧ ¬Loves(x, y)] ∨ [∃z Loves(z, x)]
5. Skolemize (y and z depend on x): ∀x [Animal(F(x)) ∧ ¬Loves(x, F(x))] ∨ Loves(G(x), x)
6. drop ∀: [Animal(F(x)) ∧ ¬Loves(x, F(x))] ∨ Loves(G(x), x)
7. distribute: **[Animal(F(x)) ∨ Loves(G(x), x)] ∧ [¬Loves(x, F(x)) ∨ Loves(G(x), x)]** — two clauses.

Steps of a resolution-refutation proof (write these in every answer)

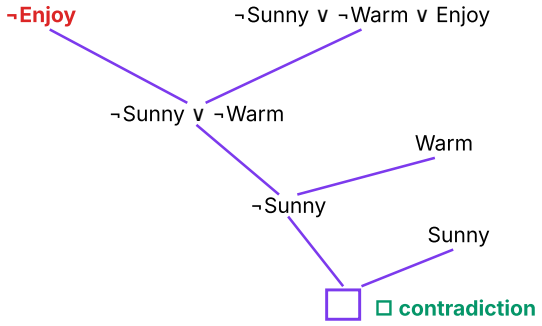
1. Convert all facts/axioms into **FOL**.
2. Convert the FOL sentences into **clause form (CNF)**.
3. **Negate the statement to be proved**, convert to clause form, and add it to the KB.
4. **Resolve** pairs of clauses (using unification), producing new clauses that logically follow; draw the **resolution graph**.

5 If the **empty clause (NIL, □)** is produced → contradiction → the original statement is **true**. The substitutions used tell *under which bindings* the goal is true (answer extraction).

Example 1 (PL): "You will enjoy"

Facts: (1) If it is sunny and warm, you will enjoy. (2) If it is raining you will get wet. (3) It is warm. (4) It is raining. (5) It is sunny. Prove: enjoy.

FOL → CNF
Sunny ∧ Warm ⇒ Enjoy → ¬(Sunny ∧ Warm) ∨ Enjoy → **¬Sunny ∨ ¬Warm ∨ Enjoy**
Raining ⇒ Wet → **¬Raining ∨ Wet**
Warm, **Raining**, **Sunny**
Negated goal: **¬Enjoy**



The negation of the conclusion leads to a complete contradiction with the given facts. **Hence proved: you will enjoy.** (Note "Raining ∨ Wet" clauses were not needed — that's normal.)

Example 2 (PL): "It will rain"

(1) The humidity is high or the sky is cloudy. (2) If the sky is cloudy, then it will rain. (3) If the humidity is high, then it is hot. (4) It is not hot. Goal: it will rain. Symbols: P = humidity is high, Q = sky is cloudy, R = it will rain, S = it is hot.

#	Statement	FOL	Clause
C1	humidity high or cloudy	P ∨ Q	P ∨ Q
C2	cloudy ⇒ rain	Q ⇒ R	¬Q ∨ R
C3	humid ⇒ hot	P ⇒ S	¬P ∨ S
C4	not hot	¬S	¬S
C5	negated goal		**¬R**

① C5 ¬R + C2 ¬Q ∨ R → **C6: ¬Q**
② C6 ¬Q + C1 P ∨ Q → **C7: P**
③ C7 P + C3 ¬P ∨ S → **C8: S**
④ C8 S + C4 ¬S → □ (empty clause)
Contradiction ⇒ ¬R is false ⇒ **It will rain**
(Slide order: PvQ + ¬QvR → PvR; + ¬PvS → RvS; + ¬S → R; + ¬R → □ — same result.)

Example 3: Socrates / "Dead dog" problem (FOL)

Socrates: C1 Human(Socrates); C2 ¬Human(x) ∨ Mortal(x); C3 ¬Mortal(Socrates) (negated goal). C1 + C2 with {x/Socrates} → Mortal(Socrates); + C3 → □. Hence Mortal(Socrates).

Dead dog: All dogs are animals; Fido is a dog; all animals will die. Prove: Fido will die.
C1 ¬dog(X) ∨ animal(X); C2 dog(fido); C3 ¬animal(Y) ∨ die(Y); C4 ¬die(fido).
C4 + C3 {Y/fido} → ¬animal(fido); + C1 {X/fido} → ¬dog(fido); + C2 → □. Hence die(fido).

Example 4: "Is John happy?" ★ Very likely

"Anyone passing his history exams and winning the lottery is happy. But anyone who studies or is lucky can pass all his exams. John did not study, but John is lucky. Anyone who is lucky wins the lottery."

FOL

∀X pass(X, history) ∧ win(X, lottery) ⇒ happy(X)

∀X ∀Y study(X) ∨ lucky(X) ⇒ pass(X, Y)

¬study(john) ∧ lucky(john)

∀X lucky(X) ⇒ win(X, lottery)

Second rule: ¬(study ∨ lucky) ∨ pass → (¬study ∨ pass) ∧ (¬lucky ∨ pass) → two clauses, standardized apart.

Clauses

1. ¬pass(X, history) ∨ ¬win(X, lottery) ∨ happy(X)
2. ¬study(Y) ∨ pass(Y, Z)
3. ¬lucky(W) ∨ pass(W, V)
4. ¬study(john) 5. lucky(john)
6. ¬lucky(U) ∨ win(U, lottery)
7. **¬happy(john)** (negated goal)

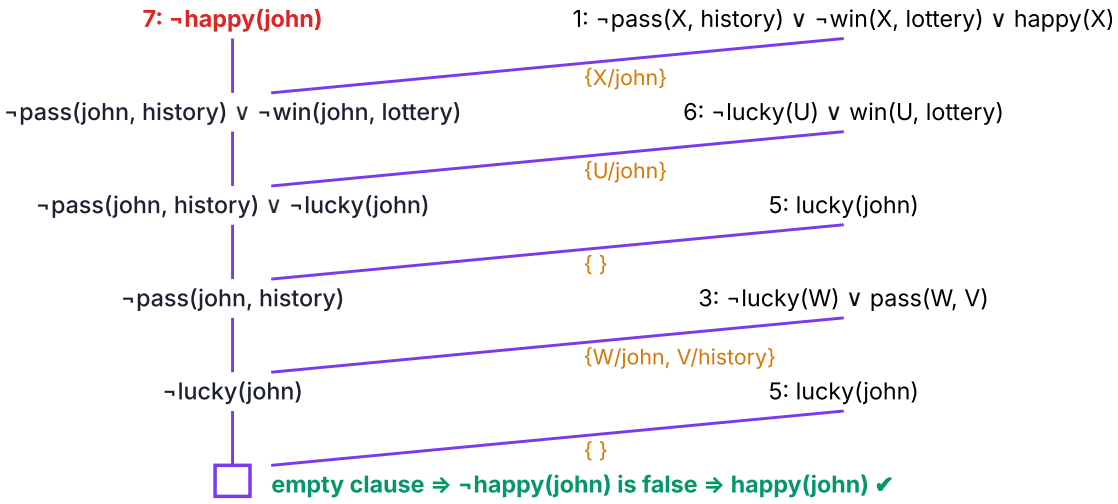


Fig 6.3 Resolution refutation for "John is happy". Left column: resolvents; right column: clause used; orange: substitution.

Example 5: "Exciting life" (answer extraction with ∃)

All people who are not poor and are smart are happy. Those people who read are not stupid. John can read and is wealthy. Happy people have exciting lives. Can anyone be found with an exciting life? Assume smart ≡ ¬stupid and wealthy ≡ ¬poor.

∀X (¬poor(X) ∧ smart(X) ⇒ happy(X)) → **poor(X) ∨ ¬smart(X) ∨ happy(X)**

∀Y (read(Y) ⇒ smart(Y)) → **¬read(Y) ∨ smart(Y)**

read(john) ∧ ¬poor(john) → **read(john)**, **¬poor(john)**

∀Z (happy(Z) ⇒ exciting(Z)) → **¬happy(Z) ∨ exciting(Z)**

Negated goal ¬∃W exciting(W) → **¬exciting(W)**

```
¬exciting(W) + ¬happy(Z)vexciting(Z)   {Z/W}
 → ¬happy(Z) + poor(X)v¬smart(X)vhappy(X)   {X/Z}
 → poor(X)v¬smart(X) + ¬read(Y)vsmart(Y)   {Y/X}
 → poor(Y)v¬read(Y) + ¬poor(john)          {john/Y}
 → ¬read(john) + read(john)  → □
Answer extraction: W = john → exciting(john)
```

Example 6: "Where is Fido?" (answer extraction)

Dog Fido goes wherever John goes: ∀X at(john, X) ⇒ at(fido, X) → **¬at(john, X) ∨ at(fido, X)**. John is at the library: **at(john, library)**. Negated query ¬∃Z at(fido, Z) → **¬at(fido, Z)**. Resolve ¬at(fido, Z) with clause 1 {X/Z} → ¬at(john, X); with at(john, library) {library/X} → □. Keep a copy of the goal at(fido, Z) and apply the same substitutions → **at(fido, library)**.

Example 7: "What does Jade eat?"

"Jade eats whatever Tom eats, and Tom eats banana. What does Jade eat?" (The slide words it as "Tom eats whatever Jade eats", but its FOL — used below — says whatever Tom eats, Jade eats.) 1. $\forall X\ \text{eat}(\text{tom}, X) \Rightarrow \text{eat}(\text{jade}, X)$ → ¬eat(tom, X) ∨ eat(jade, X); 2. eat(tom, banana); negated conclusion ¬∃X eat(jade, X) → 3. ¬eat(jade, Y). Resolve 3 with 1 {Y/X} → ¬eat(tom, Y); with 2 {banana/Y} → □. Answer: **eat(jade, banana)**.

Example 8: "Mohan will deliver an excellent lecture"

All teachers are good: T(X) ⇒ G(X) → **¬T(X) ∨ G(X)**. Anyone good and intelligent delivers excellent lectures: ∀Y G(Y) ∧ I(Y) ⇒ L(Y) → **¬G(Y) ∨ ¬I(Y) ∨ L(Y)**. Mohan is an intelligent teacher: **I(Mohan)**, **T(Mohan)**. Negated goal: **¬L(Mohan)**. ¬L(Mohan) + clause 2 {Mohan/Y} → ¬G(Mohan) ∨ ¬I(Mohan); + I(Mohan) → ¬G(Mohan); + clause 1 {Mohan/X} → ¬T(Mohan); + T(Mohan) → □. **Hence L(Mohan).**

Example 9: "John likes peanuts" by resolution

Clauses: 1. ¬Food(x) ∨ Likes(John, x) 2. Food(Apple) 3. Food(Chicken) 4. ¬Eats(y, z) ∨ Killed(y) ∨ Food(z) 5. Eats(Bill, Peanut) 6. ¬Killed(Bill) 7. ¬Eats(Bill, w) ∨ Eats(Sue, w) 8. **¬Likes(John, Peanut)**
8 + 1 {x/Peanut} → ¬Food(Peanut); + 4 {z/Peanut} → ¬Eats(y, Peanut) ∨ Killed(y); + 5 {y/Bill} → Killed(Bill); + 6 → □. Hence Likes(John, Peanut).

PROPERTIES OF RESOLUTION

Resolution is **sound** and **refutation-complete**: if a set of sentences is unsatisfiable, resolution will always be able to derive a contradiction (□). It can't be used to *generate* all consequences, but it can confirm or refute any given sentence.

EXAM TIP — RESOLUTION ANSWERS

Always show: FOL → CNF (with the step names: "eliminate implication", "move negation inside", "Skolemize", "drop ∀") → numbered clause list → negated goal clearly marked → resolution graph with the substitution written on each edge → "Empty clause obtained, hence proved". Drawing the graph neatly earns marks even if one substitution is off.

6.8 One knowledge base, three methods (summary)

KB	Facts: Student(John), Studies(John). Rule: Student(x) ∧ Studies(x) ⇒ Pass(x). Query: Pass(John)?
Forward	Start from facts → apply rule with {x/John} → Pass(John). Direction: Facts → Goal.
Backward	Start with Pass(John) → rule concluding Pass → unify x = John → sub-goals Student(John), Studies(John) → both facts → proved. Direction: Goal → Facts.
Resolution	Rule → ¬Student(x) ∨ ¬Studies(x) ∨ Pass(x); add ¬Pass(John); resolve with Student(John) → ¬Studies(John) ∨ Pass(John); with Studies(John) → Pass(John); with ¬Pass(John) → □. Direction: Refutation.

Why unification is so important: it is not a separate inference algorithm — it is the matching mechanism used by every FOL method: FC (match Student(x) with fact Student(John)), BC (match goal Pass(John) with rule head Pass(x)), resolution (match P(x) with ¬P(John) to get Q(John) from P(x) ∨ Q(x)). Unification is the bridge that lets FOL inference work with variables.

LIKELY EXAM QUESTIONS — CH 6

1. Explain Universal Instantiation and Existential Instantiation with examples. (5M)
2. What is unification? State its conditions / write the algorithm; find the MGU of the given pairs. (6–8M)
3. Explain lifting and Generalized Modus Ponens with an example. (5M)
4. Prove "West is a criminal" using forward chaining (and/or backward chaining) — draw the proof tree. (10M) ★
5. Prove "John likes peanuts" using forward / backward chaining / resolution. (10M)
6. Differentiate forward and backward chaining. (5M)
7. List the steps to convert FOL to CNF; convert the given sentence. (6M)
8. Using resolution refutation, check whether John is happy / it will rain / you will enjoy. (10M) ★

60-SECOND RECAP

UI: ∀ → any ground term (many times). EI: ∃ → new Skolem constant (once). Unification = substitution making two expressions identical (same predicate, same arity, occurs check); MGU = most general. Lifting/GMP = Modus Ponens with variables. FC = data-driven to fixed point; BC = goal-driven DFS (Prolog). Resolution = add ¬goal, CNF, resolve to □; sound & refutation-complete. CNF steps: ⇒ out, ¬ in, standardize, Skolemize, drop ∀, distribute, split.

7 Knowledge Graphs & Ontologies Self-study topic

From connected facts → meaning → reasoning → intelligent answers. Big idea: **store knowledge in a form that both humans and machines can follow**.

7.1 Why do we need a knowledge graph?

PROBLEM

Information is scattered across tables, documents and apps. Keyword search finds *words* but doesn't understand how facts are *connected*.

IDEA

Represent important things as **entities** and connect them with meaningful **relationships**. The graph becomes a network of facts.

BENEFIT

We can follow connections, combine facts, **infer new knowledge** and answer **multi-step questions**.

A table stores rows; a graph makes relationships easy to follow. Example question: "*Who teaches the course studied by Asha?*" needs following more than one relationship.

7.2 Four words to understand first ★ Very likely

Term	Meaning	Campus examples
Entity	A thing we want to describe	Asha, AIML, CSE, Dr Rao
Relationship	A named connection between two entities	studies, teaches, offeredBy
Class	A category or type	Student, Person, Course, Department
Instance	A specific member of a class	Asha is an instance of Student

MEMORY TIP

Entity = **thing** • Relationship = **connection** • Class = **category** • Instance = **member**

7.3 Knowledge graph = entities + labelled edges

DEFINITION

A **knowledge graph (KG)** is a graph-structured knowledge base in which nodes represent entities (and classes) and labelled directed edges represent relationships between them. Every arrow tells us what the connection means; read an arrow as a sentence: **Asha —studies→ AIML**.

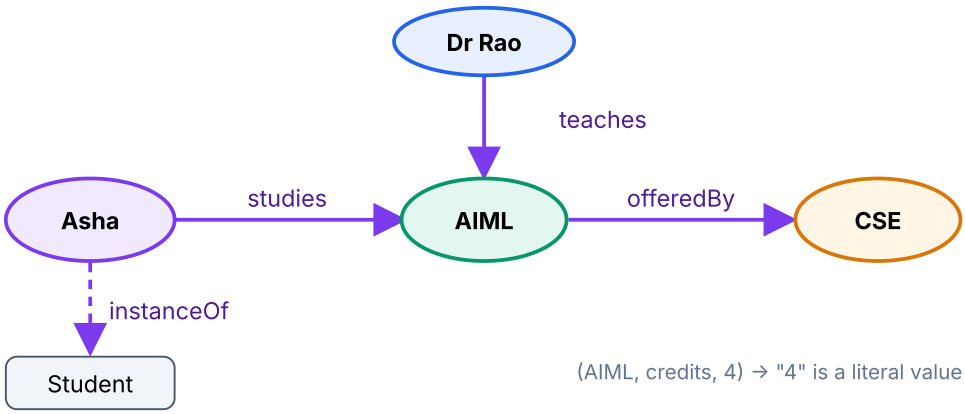


Fig 7.1 A small campus knowledge graph. Entities are nodes; each labelled arrow is one fact.

7.4 RDF triple: one fact in three parts ★ Very likely

Subject — Predicate — Object → (Asha, studies, AIML)

RDF = Resource Description Framework, the W3C standard data model for knowledge graphs.

- **Subject**: the entity the fact is about (Asha). **Predicate**: the relationship/property (studies). **Object**: the value or the other entity (AIML).
- **Literal example**: (AIML, credits, 4) — here "4" is a **value**, not another entity.
- **How triples become a graph:** repeated identifiers connect separate facts. (Asha, studies, AIML), (DrRao, teaches, AIML), (AIML, offeredBy, CSE), (Asha, instanceOf, Student) — the same AIML node links them. The graph is not drawn separately; it *emerges* because identifiers are reused.

RDF written in Turtle syntax

```
@prefix ex: <https://example.org/campus/> .
ex:Asha   ex:studies    ex:AIML .
ex:AIML   ex:offeredBy  ex:CSE .
ex:DrRao  ex:teaches    ex:AIML .
```

Turtle is a compact, human-readable syntax for RDF. `@prefix` creates a short name (ex:) for a long **IRI** (internationalized resource identifier); `ex:Asha` identifies the resource Asha in this namespace; the full stop **.** ends one statement.

7.5 Knowledge graph vs ontology ★ Very likely

KNOWLEDGE GRAPH — STORES PARTICULAR FACTS

- Asha studies AIML
- Dr Rao teaches AIML
- AIML is offered by CSE

ONTOLOGY — DEFINES MEANING, CATEGORIES & RULES

- Student is a subclass of Person
- "studies" has domain Student
- "studies" has range Course

DEFINITION — ONTOLOGY

An **ontology** is a formal, explicit specification of the vocabulary of a domain: its classes, properties (relations), the hierarchy among them, and the logical constraints/rules that give them meaning. Standards: RDFS and OWL.

Graph = facts | Ontology = vocabulary + semantics

Classes, instances and properties

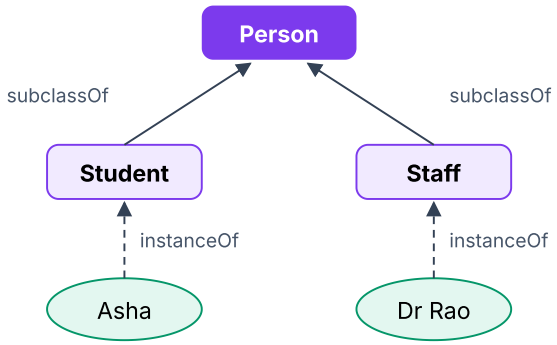


Fig 7.2 Ontology hierarchy (classes on top, instances at bottom)

Class hierarchy: Student ⊑ Person, Staff ⊑ Person (⊑ = "is a subclass of").

Instances: Asha ∈ Student, Dr Rao ∈ Staff.

Property examples: studies(Asha, AIML) — an object property linking two entities; credits(AIML, 4) — a data property linking an entity to a literal.

Semantic relationships an ontology can express: subclass (is-a), instance-of, part-of, domain, range, inverse, symmetric/transitive properties, disjoint classes.

7.6 Ontology-based reasoning (rule-based inference) ★ Very likely

Reasoning = known facts + semantic rules → additional knowledge. The ontology lets a **reasoner** derive facts that were never explicitly stored (they are *entailed*).

EXAMPLE 1 — SUBCLASS INFERENCE

Given 1: $\text{Student} \sqsubseteq \text{Person}$ (Student is a subclass of Person). Given 2: $\text{Student}(\text{Asha})$. ⇒ **Inferred fact: $\mathbf{Person(Asha)}$**. In FOL this is just $\forall x\ \text{Student}(x) \Rightarrow \text{Person}(x)$ + Modus Ponens — rule-based inference.

EXAMPLE 2 — DOMAIN, RANGE AND INVERSE: ONE TRIPLE, THREE INFERENCES

Stored fact: $\text{studies}(\text{Asha}, \text{AIML})$

Ontology rule	Inference
domain(studies) = Student	Student(Asha)
range(studies) = Course	Course(AIML)
inverse(studies) = hasStudent	hasStudent(AIML, Asha)

Important: in RDFS/OWL, domain and range are inference rules — not merely form-validation checks.

7.7 Open-world assumption: missing ≠ false

WHAT IS STORED?

"Asha studies AIML." Nothing is stored about whether Ravi studies AIML.

WHAT CAN WE CONCLUDE?

~~"Ravi does not study AIML."~~ ✘

Correct: **"We do not know from the current graph."** ✔

Open world (OWA): the graph may be incomplete, so the absence of a fact does not prove its opposite (used by RDF/OWL). Contrast with the **closed-world assumption** (databases, Prolog): anything not stored is false.

7.8 Question answering by following graph paths

Multi-step questions become **graph pattern matching**. "Who teaches a course studied by Asha?"

1. Match **Asha —studies→ ?course** → result: AIML.
2. Match **?teacher —teaches→ AIML** → result: Dr Rao.
3. Answer: **Dr Rao**, supported by the two matching relationships (the shared course connects the patterns).

SPARQL: asking the graph a question

```
PREFIX ex: <https://example.org/campus/>
SELECT ?teacher WHERE {
   ex:Asha   ex:studies  ?course .
   ?teacher  ex:teaches  ?course .
}
```

SPARQL is the query language for RDF. Variables begin with **?**. `SELECT ?teacher` returns the value bound to ?teacher. ?course appears in both patterns, so it **joins** the two facts (like a shared variable in FOL unification). Result here: Dr Rao.

7.9 Where are knowledge graphs used?

Area	Use
Search & QA	Connect people, places, topics and facts to improve answers (e.g. search-engine info panels)
Recommendation	Link users, items, categories and preferences
Healthcare	Connect diseases, symptoms, drugs and biomedical concepts
Enterprise	Connect employees, products, documents and business processes
Education	Connect students, courses, concepts, outcomes and resources
AI agents	Retrieve structured evidence before generating an answer (grounding, fewer hallucinations)

Wikidata — a real public knowledge graph. Entities use **Q-identifiers**; properties use **P-identifiers** (e.g. **P31 = "instance of"**, **P279 = "subclass of"**). SPARQL can follow relationships and class paths. Display names are for people; stable identifiers are for reliable machine linking.

7.10 How an intelligent agent uses a knowledge graph

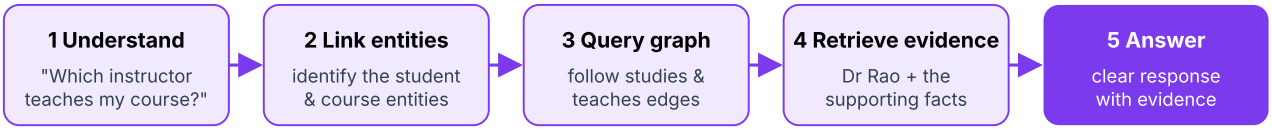


Fig 7.3 KG-powered question answering pipeline. If an entity name is ambiguous, the agent should ask for clarification rather than guess.

CLASS Q&A (EXIT TICKET) — ANSWER THESE

1. In (Asha, studies, AIML), identify subject, predicate and object. → Subject Asha, predicate studies, object AIML.
2. Difference between a knowledge graph and an ontology? → KG stores particular facts; ontology defines classes, properties, meaning and rules.
3. If Student ⊑ Person and Student(Asha), what can we infer? → Person(Asha).
4. Why does a missing triple not automatically mean "false"? → Open-world assumption: the graph may be incomplete.
5. In the SPARQL example, why is ?course used in both triple patterns? → The shared variable joins the two facts.
6. Explain "knowledge graph" in one sentence using the words entity and relationship. → A knowledge graph is a network of entities connected by labelled relationships.

ONE-SLIDE RECAP (5 LINES)

1 **Knowledge graph** connects entities using labelled relationships. 2 **RDF triple** represents one fact as subject–predicate–object. 3 **Ontology** defines classes, properties, meaning and logical constraints. 4 **Reasoning** derives implicit facts from explicit facts + ontology rules. 5 **SPARQL / agent** retrieves connected facts to answer questions. *Connected facts become more useful when their meaning is explicit.*

8 Reasoning under Uncertainty

Logic says TRUE or FALSE. The real world says "probably". This chapter teaches how an agent reasons with **degrees of belief** using probability.

8.1 What is uncertainty and why does AI need to handle it? ★ Very likely

With certain knowledge we can write a rule A → B. But if we are **not sure whether A is true**, we cannot use such a rule — this situation is called **uncertainty**. In classical logic "Ravi is a student" is simply $\text{Student}(\text{Ravi}) = \text{True}$; but for "Will it rain tomorrow?" we can't say True or False — instead $P(\text{RainTomorrow}) = 0.8$: the agent believes there is an 80% chance of rain.

DEFINITION

Uncertainty is an agent's inability to reach a definite decision about the truth of a proposition. **Probabilistic reasoning** is a way of knowledge representation where we apply probability to indicate the uncertainty in knowledge — combining probability theory with logic. Probabilities summarize the agent's beliefs relative to the available evidence.

WHY UNCERTAINTY OCCURS

- **Ignorance / laziness** — too many rules or facts to list; incomplete theory
- **Partial observability** — the agent can't see everything
- **Noisy / unreliable** sensors, experimental errors, unreliable sources
- **Nondeterminism** — outcomes of actions are unpredictable
- **Complexity**; conditions (temperature, environment) vary

NEED FOR UNCERTAINTY HANDLING

- Reason when outcomes are unpredictable
- Combine prior knowledge with newly observed evidence
- Make useful decisions even with missing information
- Avoid forcing every situation into TRUE/FALSE
- Examples: medical diagnosis, weather prediction, robot localization, spam filtering, matches between teams

EXAMPLE — AUTONOMOUS CAR

Camera: possibly a pedestrian · Radar: object detected · GPS: approximate position · Weather & road: uncertain. The car must still decide: Camera detects object → is it a pedestrian? → maybe 90% → should it brake? → decision. A probabilistic system reasons with $P(\text{Pedestrian} \mid \text{Evidence}) = 0.90$ and uses that to choose an action. Two ways to solve such problems: **Bayes' rule** and **Bayesian statistics**.

8.2 Basic probability concepts

$$P(A) = \frac{\text{number of desired outcomes}}{\text{total number of outcomes}}$$

$0 \le P(A) \le 1 \cdot P(\neg A) + P(A) = 1$

- **Event**: each possible outcome of a variable.
- **Sample space**: the collection of all possible events.
- **Random variables**: represent events and objects in the real world.
- **Prior probability**: probability computed *before* observing new information.
- **Posterior probability**: probability computed *after* all evidence is taken into account (prior + new information).
- **Sum rule**: $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$.

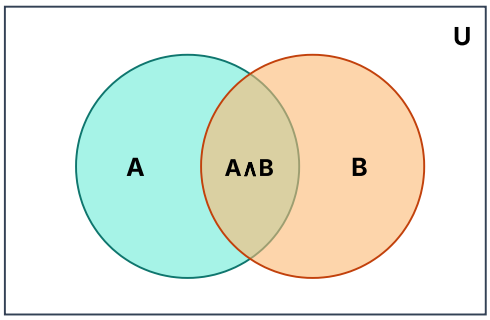


Fig 8.1 P(A|B): we zoom into circle B and ask what fraction of it is also A.

8.3 Conditional probability ★ Very likely

DEFINITION

Conditional probability P(A|B) is the probability of event A given that event B has already occurred ("probability of A under the condition B").

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)} \quad (P(B) > 0)$$

$P(A \wedge B)$ = joint probability of A and B; $P(B)$ = marginal probability of B

EXAMPLE 1 — SUSAN'S TESTS

P(passing both tests) = 0.6; P(passing the first) = 0.8. P(second | first)?

$$P(\text{second} \mid \text{first}) = \frac{P(\text{first} \wedge \text{second})}{P(\text{first})} = \frac{0.6}{0.8} = \boxed{0.75}$$

EXAMPLE 2 — ENGLISH & MATHS

70% of students like English, 40% like English *and* Maths. What % of English-likers also like Maths? A = likes Maths, B = likes English.

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{0.4}{0.7} = \boxed{57\%}$$

Why it matters in AI: an agent observes evidence (Fever = Yes, Cough = Yes) and wants $P(\text{Disease} \mid \text{Fever}, \text{Cough})$ — evidence changes the probability of the hypothesis. Fundamental to medical diagnosis, spam detection, fault diagnosis, speech recognition, recommendation, fraud detection and intrusion detection.

Product rule

$$P(A \wedge B) = P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$$

Both describe the same joint event $A \wedge B$ — this leads directly to Bayes' theorem.

8.4 Bayes' theorem ★ Very likely

Also called **Bayes' rule, Bayes' law** or **Bayesian reasoning**; named after British mathematician **Thomas Bayes**. It relates the conditional and marginal probabilities of two events, i.e. it lets us compute P(B|A) when we know P(A|B) — it "reverses" a conditional probability. It is the basis of Bayesian inference and of most modern AI systems for probabilistic inference.

WHY DO WE NEED IT?

Doctors know $P(\text{Symptom} \mid \text{Disease})$ (how often the disease causes the symptom) — but diagnosis needs $P(\text{Disease} \mid \text{Symptom})$. These are **not** the same. Bayes' theorem converts one into the other.

Derivation (write this for 3 marks)

Product rule: $P(A \wedge B) = P(A \mid B)\,P(B)$ … (1) and $P(A \wedge B) = P(B \mid A)\,P(A)$ … (2)
Equating the right-hand sides of (1) and (2): $P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$
Dividing by $P(B)$: **$P(A \mid B) = P(B \mid A)\,P(A) / P(B)$** … Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

Posterior = Likelihood × Prior / Evidence

Term	Name	Meaning (email example: A = Spam, B = contains "offer")
P(A|B)	**Posterior**	Probability of hypothesis A after observing evidence B — what we want
P(B|A)	**Likelihood**	Probability of the evidence assuming the hypothesis is true (spam emails containing "offer")
P(A)	**Prior**	Probability of the hypothesis before seeing evidence (overall fraction of spam)
P(B)	**Evidence / marginal**	Pure probability of the evidence (any email containing "offer")

General form with the theorem of total probability

$$P(B) = \Sigma_i\, P(B \mid A_i)\, P(A_i) \quad \Rightarrow \quad P(A_i \mid B) = \frac{P(B \mid A_i)\, P(A_i)}{\Sigma_k\, P(B \mid A_k)\, P(A_k)}$$

$A_1, \ldots, A_n$ mutually exclusive and exhaustive ($\Sigma P(A_i) = 1$)

Mitchell's (ML textbook) notation

$$P(h \mid D) = \frac{P(D \mid h)\, P(h)}{P(D)}$$

h = hypothesis, D = observed training data. P(h): prior · P(D): marginal (prior probability of seeing D) · P(D|h): likelihood · P(h|D): posterior.

P(h|D) increases with P(h) and with P(D|h); P(h|D) **decreases as P(D) increases** — the more probable D is independent of h, the less evidence D gives in support of h.

Solved Bayes problems ★ Very likely

1 · MENINGITIS & STIFF NECK

M = meningitis, S = stiff neck. P(S|M) = 0.7, P(M) = 1/50000, P(S) = 0.01.

$$P(M|S) = \frac{P(S|M)\,P(M)}{P(S)} = \frac{0.7 \times (1/50000)}{0.01} = \frac{0.000014}{0.01} = \boxed{0.0014}\ (1 \text{ in } \sim 714)$$

Lesson: even though meningitis causes a stiff neck 70% of the time, a stiff-neck patient has a tiny chance of meningitis because the disease is extremely rare. A strong symptom does not mean a high probability of disease — the prior matters.

2 · SPAM AND THE WORD "OFFER"

P(S) = 0.20, P(W|S) = 0.80, P(W) = 0.40.

$$P(S|W) = \frac{0.80 \times 0.20}{0.40} = \boxed{0.40} \rightarrow$$

$$P(\text{Spam}|\text{Offer}) = 40\%$$

3 · FACE CARD → KING

One card from 52. Given it's a face card (J, Q, K → 12 cards), P(King)?

$$P(K|F) = \frac{P(F|K)\,P(K)}{P(F)} = \frac{1 \times 4/52}{12/52} = \boxed{1/3 \approx 33.33\%}$$

(Every King is a face card so P(F|K) = 1; out of every 3 face cards 1 is a King.)

4 · EXERCISE FROM CLASS — LIVER DISEASE & ALCOHOLISM

A = patient has liver disease, B = patient is alcoholic. 10% of patients have liver disease, 5% are alcoholics, and among liver-disease patients 7% are alcoholics. P(A|B)?

$$P(A) = 0.10,\ P(B) = 0.05,\ P(B|A) = 0.07 \rightarrow P(A|B) = \frac{0.07 \times 0.10}{0.05} = \boxed{0.14}\ (14\%)$$

Applications of Bayes' theorem in AI: medical diagnosis (update disease probability from symptoms/tests), robot localization (update belief about position from sensors), weather prediction, fault diagnosis (most likely cause of a failure), classification (P(class | features)).

8.5 MAP — Maximum A Posteriori hypothesis ★ Very likely

A **hypothesis** h is a possible explanation/assumption about something we want to predict. Given a set of candidate hypotheses H = {h_1, h_2, ...} and observed data D, we want the hypothesis that is **most likely to be true given D**.

$$h_{MAP} \equiv \text{argmax}_{h \in H}\ P(h|D) = \text{argmax}_{h \in H}\ \frac{P(D|h)\,P(h)}{P(D)} = \text{argmax}_{h \in H}\ P(D|h)\,P(h)$$

P(D) is the same for every hypothesis, so it can be dropped. (If all priors are equal, this reduces to the maximum-likelihood hypothesis h_{ML} = argmax P(D|h).)

BOX OF BALLS (CLASS EXAMPLE)

Hypotheses: h_1 = box with mostly red balls, h_2 = mostly blue, h_3 = equal red and blue. Observed D = {red, red, red}. Posteriors computed with Bayes: $P(h_1|D) = 0.70$ (most likely), $P(h_2|D) = 0.10$, $P(h_3|D) = 0.20$ → $\mathbf{h_{MAP} = h_1}$, the most probable explanation of the data.

THE CANCER TEST PROBLEM (MITCHELL) — MOST ASKED

Two hypotheses: patient has cancer (*cancer*) or not (¬*cancer*). The lab test returns a correct positive in only **98%** of cases where the disease is present, and a correct negative in only **97%** of cases where it is not. **0.008** of the population has this cancer. A new patient tests **positive**. Diagnose using MAP.

Given	Complement
P(cancer) = 0.008	P(¬cancer) = 1 − 0.008 = 0.992
P(+ | cancer) = 0.98	P(− | cancer) = 0.02
P(− | ¬cancer) = 0.97	P(+ | ¬cancer) = 0.03

$P(+ \mid \text{cancer})\, P(\text{cancer}) = 0.98 \times 0.008 = \mathbf{0.0078}$

$P(+ \mid \neg\text{cancer})\, P(\neg\text{cancer}) = 0.03 \times 0.992 = \mathbf{0.0298}$

$0.0298 > 0.0078 \Rightarrow$ **$h_{MAP} = \neg$cancer** — the patient with a positive test is diagnosed as **not** having cancer.

Exact posterior (normalize): $P(\text{cancer} \mid +) = 0.0078 / (0.0078 + 0.0298) = \mathbf{0.21}$; $P(\neg\text{cancer} \mid +) = 0.79$.

Negative test result (second slide problem):

$P(- \mid \text{cancer})\, P(\text{cancer}) = 0.02 \times 0.008 = 0.00016 \quad P(- \mid \neg\text{cancer})\, P(\neg\text{cancer}) = 0.97 \times 0.992 = 0.96224 \Rightarrow h_{MAP} = \neg\text{cancer}$

$P(-) = 0.00016 + 0.96224 = 0.9624 \quad \rightarrow P(\text{cancer} \mid -) = 0.00016 / 0.9624 = 0.000166 \approx \mathbf{0.0166\%}, \quad P(\neg\text{cancer} \mid -) = 0.96224 / 0.9624 \approx \mathbf{99.98\%}$ → not having cancer.

Why is a positive test still "no cancer"? The disease is so rare (prior 0.008) that false positives from the huge healthy population (3% of 99.2%) outnumber true positives.

EXAM TIP — BAYES/MAP NUMERICALS

(1) Write all given probabilities **and their complements** in a table. (2) Write the formula. (3) Compute $P(D|h)P(h)$ for each hypothesis. (4) Pick the largest = MAP. (5) Normalize if asked for the actual probability (divide each by their sum; they must add to 1).

8.6 Bayesian networks ★ Very likely

DEFINITION

A **Bayesian network** (Bayes network, belief network, decision network, Bayesian model) is a probabilistic graphical model that represents a set of random variables and their conditional dependencies using a Directed Acyclic Graph (DAG), with a conditional probability distribution attached to each node.

- Each **node** = a random variable (continuous or discrete).

- Each **arc (directed arrow)** = a direct causal relationship / probabilistic dependence. If X → Y, X is a **parent** of Y. No arrow between two nodes means they are (conditionally) independent.
- The graph has **no directed cycles**.
- Each node X_i has a **CPT (conditional probability table)** giving $P(X_i \mid Parents(X_i))$; nodes without parents just have a prior.
- Two components: **causal component** (structure: the DAG) + **actual numbers** (the CPTs). Bayesian network = structure + probability values. The generalized form for decision problems is an **influence diagram**.
- Uses: prediction, anomaly detection, diagnostics, automated insight, reasoning, time-series prediction, decision making under uncertainty.

Why not just use one big probability table?

The full joint distribution of n Boolean variables needs 2^n entries: 2^{10} = 1024, 2^{20} = 1,048,576, 2^{100} — enormous. A Bayesian network exploits **independence and conditional independence** to store far fewer numbers.

Joint probability: chain rule vs Bayesian network

Chain rule (always true): $P(x_1, \ldots, x_n) = P(x_1 \mid x_2, \ldots, x_n)\, P(x_2 \mid x_3, \ldots, x_n) \ldots P(x_n)$
e.g. $P(R, S, W) = P(R \mid S, W)\, P(S \mid W)\, P(W)$ — hard to use: each variable depends on all others.

Bayesian network: $P(x_1, \ldots, x_n) = \prod_{i=1..n} P(x_i \mid Parents(X_i))$
Each variable depends only on its parents. Rain → Sprinkler, both → WetGrass: $P(R, S, W) = P(R) \cdot P(S|R) \cdot P(W|S,R)$.

Conditional independence

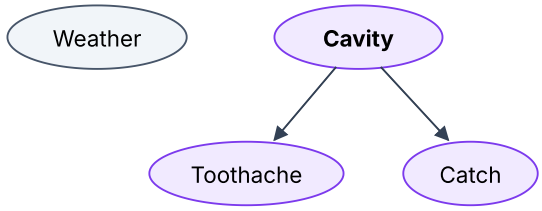


Fig 8.2 Weather is independent of the rest; Toothache and Catch are conditionally independent given Cavity.

Once we know whether there is a cavity, toothache tells us nothing more about whether the probe catches. So

$$P(Toothache, Catch, Cavity) = P(Toothache \mid Cavity) \cdot P(Catch \mid Cavity) \cdot P(Cavity)$$

instead of one large table. This is the key idea behind Bayesian networks (and Naive Bayes).

CPT example and the simplest joint computation

Cloudy → Rain: P(Rain | Cloudy) = 0.8, P(Rain | ¬Cloudy) = 0.2.

Cloudy	P(Rain)
True	0.8
False	0.2

This is a **Conditional Probability Table**. Each row must sum to 1 with its complement column (P(¬Rain|Cloudy) = 0.2, etc.).

CLOUDY → RAIN → WETGRASS

P(Cloudy) = 0.5, P(Rain|Cloudy) = 0.8, P(WetGrass|Rain) = 0.9.

$$P(C, R, W) = P(C)\, P(R|C)\, P(W|R) = 0.5 \times 0.8 \times 0.9 = \boxed{0.36}$$

Example — A, B, C, D network (class)

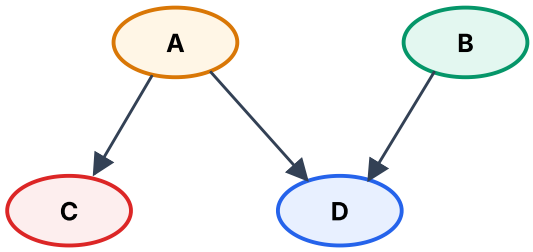
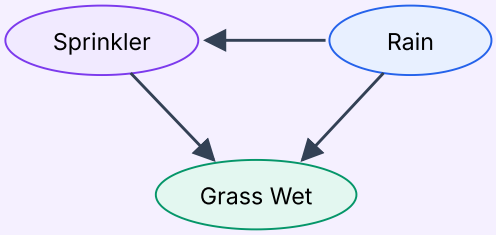


Fig 8.3 A, B: no parents (unconditional, independent, evidence/parent nodes). C, D: conditional, dependent, hypothesis/child nodes.

Given	Value	Given	Value
P(A)	0.3	P(D | A, B)	0.7
P(B)	0.6	P(D | A, ¬B)	0.4
P(C | A)	0.4	P(D | ¬A, B)	0.2
P(C | ¬A)	0.2	P(D | ¬A, ¬B)	0.01

$P(A, B, C, D) = P(D \mid A, B) \times P(C \mid A) \times P(B) \times P(A)$
e.g. all true: $0.7 \times 0.4 \times 0.6 \times 0.3 = \mathbf{0.0504}$
$P(\neg A, B, \neg C, D) = 0.2 \times 0.8 \times 0.6 \times 0.7 = \mathbf{0.0672}$

PROBLEM — RAIN, SPRINKLER, GRASS WET: FIND P(W) ★



R	P(R)	R	P(S|R)
T	0.2	F	0.4
F	0.8	T	0.01

S	R	P(W | S, R)
¬S	¬R	0.0
¬S	R	0.8
S	¬R	0.9
S	R	0.99

Step 1 complements: $P(\neg R) = 0.8$; $P(\neg S|R) = 0.99$; $P(\neg S|\neg R) = 0.60$.

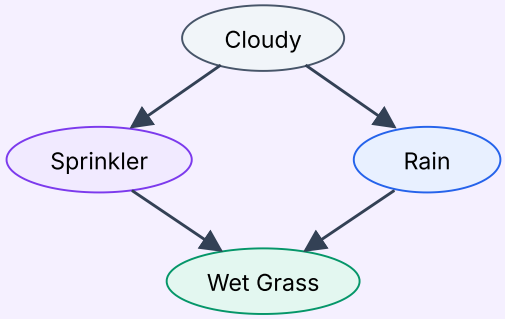
Step 2 law of total probability: $P(W) = \Sigma_S \Sigma_R P(W \mid S, R)\, P(S \mid R)\, P(R)$

Step 3 $P(W|S,R)P(S|R)P(R) = 0.99 \times 0.01 \times 0.2 = 0.00198$ | $P(W|S,\neg R)P(S|\neg R)P(\neg R) = 0.90 \times 0.4 \times 0.8 = 0.28800$

$P(W|\neg S,R)P(\neg S|R)P(R) = 0.80 \times 0.99 \times 0.2 = 0.15840$ | $P(W|\neg S,\neg R)P(\neg S|\neg R)P(\neg R) = 0.00 \times 0.60 \times 0.8 = 0$

Step 4 $P(W) = 0.00198 + 0.28800 + 0.15840 + 0 =$ **0.44838 ≈ 0.448**

PROBLEM 3 (25 PTS) — CLOUDY, SPRINKLER, RAIN, WET GRASS: P(W) AND P(S|W)



P(C) = 0.5. P(S|C): t 0.10, f 0.50. P(R|C): t 0.80, f 0.20. P(W|S,R): tt 0.99, tf 0.90, ft 0.90, ff 0.00.
(Hint: requires summing out over a few variables.)

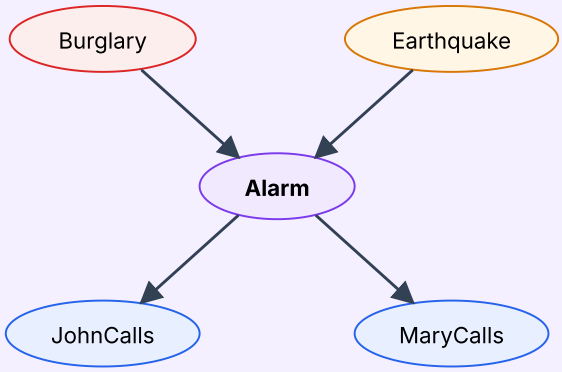
P(W) = $\Sigma_{c,s,r}$ P(W|s,r) P(s|c) P(r|c) P(c)
C = T: 0.99·0.10·0.80 + 0.90·0.10·0.20 + 0.90·0.90·0.80 + 0·0.90·0.20 = 0.0792 + 0.0180 + 0.6480 + 0 = 0.7452 → × 0.5 = 0.3726
C = F: 0.99·0.50·0.20 + 0.90·0.50·0.80 + 0.90·0.50·0.20 + 0 = 0.0990 + 0.3600 + 0.0900 = 0.5490 → × 0.5 = 0.2745
P(W) = 0.3726 + 0.2745 = **0.6471**

P(S|W) = P(S, W) / P(W). First P(S) = 0.10×0.5 + 0.50×0.5 = 0.30.
P(S, W) = Σ_c P(c) P(S|c) Σ_r P(r|c) P(W|S,r) = 0.5·0.10·(0.80·0.99 + 0.20·0.90) + 0.5·0.50·(0.20·0.99 + 0.80·0.90)
= 0.05 × 0.972 + 0.25 × 0.918 = 0.0486 + 0.2295 = 0.2781 ⇒ P(S|W) = 0.2781 / 0.6471 = **0.4298 ≈ 0.43**

NOTE ABOUT THE CLASS SLIDE

The slide computes P(W|S) = Σ P(W|S,r) P(r|c) P(c) = 0.945 and gets P(S|W) = 0.945 × 0.30 / 0.6471 ≈ **0.438**. That shortcut uses P(c) where it should use P(c|S) (knowing the sprinkler is on makes "cloudy" less likely: P(C|S) = 0.05/0.30 = 1/6). The exact answer is **0.430**. In the exam, show the method; if your teacher expects 0.438, mention both.

THE BURGLARY NETWORK (TEXTBOOK FIG 14.2)



B	E	P(A|B,E)
t	t	0.95
t	f	0.94
f	t	0.29
f	f	0.001

P(B) = 0.001, P(E) = 0.002.
P(J|A) = 0.90, P(J|¬A) = 0.05.
P(M|A) = 0.70, P(M|¬A) = 0.01.

Meaning: a burglary or an earthquake can set off the alarm; if the alarm sounds, John and/or Mary may call. JohnCalls and MaryCalls are conditionally independent given Alarm.

"Alarm sounded, both called, but no burglary and no earthquake":
P(j, m, a, ¬b, ¬e) = P(j|a) P(m|a) P(a|¬b,¬e) P(¬b) P(¬e) = 0.90 × 0.70 × 0.001 × 0.999 × 0.998 = **0.000628**

EXAM TIP — BAYESIAN NETWORK NUMERICALS

(1) Draw the DAG. (2) Write the factorization P = ∏ P(node | parents) for exactly the variables asked. (3) Fill each factor from the CPT, using 1 − p for negations. (4) For a marginal like P(W), sum the joint over every combination of the hidden variables — make a small table so you don't miss a row.

8.7 Naive Bayes classifier (Mitchell's formulation) ★ Very likely

One highly practical Bayesian learning method. Each instance x is described by a conjunction of attribute values ⟨a_1, a_2, …, a_n⟩, and the target function f(x) takes values from a finite set V. Given training

examples and a new instance, predict its target value (classification).

$$v_{MAP} = \text{argmax}_{v_j \in V} P(v_j \mid a_1, \ldots, a_n) = \text{argmax}_{v_j} \frac{P(a_1, \ldots, a_n \mid v_j)\, P(v_j)}{P(a_1, \ldots, a_n)} = \text{argmax}_{v_j} P(a_1, \ldots, a_n \mid v_j)\, P(v_j)$$

Naive Bayes assumption (attributes conditionally independent given the class): $P(a_1, \ldots, a_n \mid v_j) = \prod_i P(a_i \mid v_j)$

$$v_{NB} = \text{argmax}_{v_j \in V} P(v_j) \prod_i P(a_i \mid v_j)$$

PLAYTENNIS — CLASSIFY ⟨OUTLOOK = SUNNY, TEMPERATURE = COOL, HUMIDITY = HIGH, WIND = STRONG⟩ ★ VERY LIKELY

Day	Outlook	Temp	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Outlook	Yes	No	Temp	Yes	No
sunny	2/9	3/5	hot	2/9	2/5
overcast	4/9	0	mild	4/9	2/5
rain	3/9	2/5	cool	3/9	1/5
Humidity	**Yes**	**No**	**Wind**	**Yes**	**No**
high	3/9	4/5	strong	3/9	3/5
normal	6/9	1/5	weak	6/9	2/5

Priors: P(yes) = 9/14 = 0.64, P(no) = 5/14 = 0.36.

v_{NB}(yes) = P(yes) P(sunny|yes) P(cool|yes) P(high|yes) P(strong|yes) = (9/14)(2/9)(3/9)(3/9)(3/9) = **0.0053**

v_{NB}(no) = P(no) P(sunny|no) P(cool|no) P(high|no) P(strong|no) = (5/14)(3/5)(1/5)(4/5)(3/5) = **0.0206**

0.0206 > 0.0053 ⇒ **PlayTennis = No**

Normalize: P(no | x) = 0.0206 / (0.0206 + 0.0053) = **0.795**; P(yes | x) = **0.205**; check 0.205 + 0.795 = 1 ✔

STOLEN VEHICLES — CLASSIFY (RED, SUV, DOMESTIC)

#	Color	Type	Origin	Stolen?
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	No
3	Red	Sports	Domestic	Yes
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	No
7	Yellow	SUV	Imported	Yes
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	Sports	Imported	Yes

P(Yes) = 5/10 = 0.5, P(No) = 0.5.

	Yes	No
Red	3/5	2/5
SUV	1/5	3/5
Domestic	2/5	3/5

$P(\text{Yes} \mid x) \propto (5/10)(3/5)(1/5)(2/5) = 3/125 = \mathbf{0.024}$ $P(\text{No} \mid x) \propto (5/10)(2/5)(3/5)(3/5) = 9/125 = \mathbf{0.072}$

$0.072 > 0.024 \Rightarrow$ **Not stolen**

More Naive Bayes (email example with Laplace smoothing, types of NB, zero-frequency, Unit III view) is in **Chapter 12**. A related 10-mark question from the slides: estimate conditional probabilities of {colour, legs, height, smelly} for species {M, H} and classify (Colour = Green, Legs = 2, Height = Tall, Smelly = No) — solved in Chapter 12.

LIKELY EXAM QUESTIONS — CH 8

1. What is uncertainty? Why is uncertainty handling needed in AI? Give reasons and examples. (5M)
2. Define conditional probability; solve a small problem (Susan / English–Maths). (4M)
3. State and derive Bayes' theorem; explain prior, likelihood, posterior, evidence. (6M) ★
4. Solve: meningitis / cancer test using MAP. (6–10M) ★
5. What is a Bayesian network? Explain its components, CPT and joint probability with an example. (8–10M) ★
6. Given a Bayesian network, compute P(W) / a joint probability. (10M)
7. Explain the Naive Bayes classifier and classify the PlayTennis / stolen-car instance. (10M) ★

60-SECOND RECAP

$P(A|B) = P(A \wedge B)/P(B)$. Bayes: posterior = likelihood × prior / evidence. $h_{MAP} = \text{argmax}\ P(D|h)P(h)$. Cancer: 0.0078 vs 0.0298 → ¬cancer. Bayesian network = DAG + CPTs; $P(x_{1..n}) = \prod P(x_i|\text{parents})$. Full joint needs 2^n numbers; BN uses conditional independence. Naive Bayes: $v_{NB} = \text{argmax}\ P(v)\prod P(a_i|v)$; PlayTennis → No (0.795).

9 Introduction to Machine Learning

Text books: Tom M. Mitchell, *Machine Learning*; Peter Harrington, *Machine Learning in Action* (Ch 1.1–1.3). Topics: what is ML, key terminology, key tasks, well-posed learning problems, types of ML — plus concept learning (Find-S, Candidate-Elimination) taught in class.

9.1 What is machine learning? ★ Very likely

THREE DEFINITIONS — LEARN ALL THREE, WRITE TWO

1. **Arthur Samuel (1959):** "Machine Learning is the field of study that gives computers the capability to learn without being explicitly programmed."
2. **Tom M. Mitchell (1997):** "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."
3. **Svensson & Söderberg (2008):** "ML is concerned with the design and development of algorithms and techniques that allow computers to 'learn'. The major focus of ML research is to extract information from data automatically, by computational and statistical methods. It is thus closely related to data mining and statistics."

In short: **learning = improving with experience at some task** — improve over task T, with respect to performance measure P, based on experience E. The machine receives data as input and uses an algorithm to formulate answers; ML combines data with statistical tools to predict an output, which is then used for actionable insights. ML is an application (sub-field) of Artificial Intelligence; its core objective is **learning and inference**.

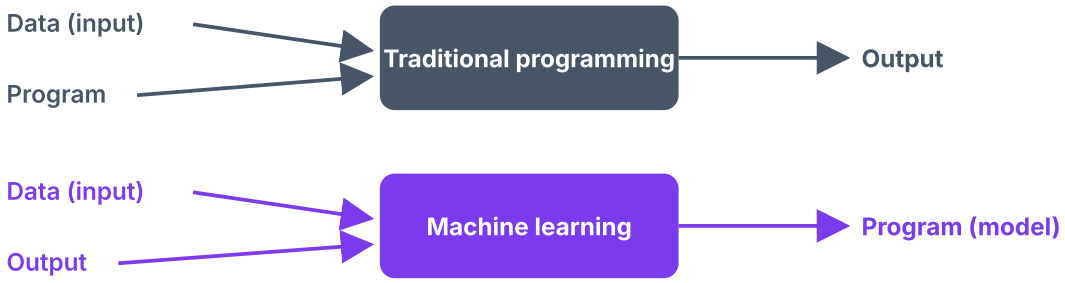


Fig 9.1 Traditional programming: data + program → output. Machine learning: data + output (examples) → program (the learned model).

Why machine learning? / Why is it important?

- Recent progress in algorithms and theory; growing flood of online data; huge memory & compute power; industry demand; less or no human intervention; better decision making and predictions.
- Computers capture terabytes of data daily; its size and complexity mean humans can't extract useful information. We can reduce dimensions, or let computers (which don't get bored and don't find high dimensions hard) do the work. A 2-D projection of a 3-D object hides information; glyphs (size/colour) help only up to a few dimensions, not 100.
- Some tasks can be defined only by examples (recognizing people).
- Relationships and correlations hidden in large data can be found (data mining).
- Human-designed machines often don't work as well as desired in their environment.
- Knowledge for some tasks is too large to encode by hand (medical diagnosis).
- Environments change over time; new knowledge keeps being discovered — redesigning by hand is hard.

When to learn?

When **human expertise does not exist** (navigating on Mars); when **humans can't explain their expertise** (speech recognition); when **solutions change over time** (routing in networks); when solutions must be adapted to particular

cases (user biometrics).

Learning — key ideas from the slides

- Machines learn from **data** ("experience"). Early AI = **symbolic processing** (computer manipulates symbols reflecting the environment); ML methods are sometimes called **subsymbolic** (no symbolic manipulation).
- ML is about making computers **modify or adapt their actions** (predictions or controlling a robot) so that they get more accurate, accuracy = how well the actions reflect the correct ones.
- **Scrabble example:** you beat the computer at first, but after many games it beats you every time and can use the same strategies against other players → generalisation.
- ML merges ideas from neuroscience and biology, statistics, mathematics and physics; data mining shifted emphasis back to efficient algorithms.
- **Computational complexity** has two parts: complexity of **training** (rare, not time-critical) and of **applying** the trained model (must be fast — many test points).
- **What to learn? Direct learning** (discriminative / model-free): learn a function mapping input → output. **Indirect learning** (generative / model learning): learn a model of the domain, then use it to answer questions.

Areas of influence (ML is interdisciplinary)

Field	Contribution / question
Statistics	How best to use samples drawn from unknown probability distributions to decide from which distribution a new sample is drawn?
Brain models	Non-linear elements with weighted inputs (artificial neural networks) as simple models of biological neurons
Adaptive control theory	Controlling a process with unknown parameters that must be estimated during operation
Psychology	Modelling human performance on learning tasks
Artificial intelligence	Algorithms that acquire knowledge humans acquire, at least as well as humans
Evolutionary models	Modelling biological evolution to improve program performance
Information theory, philosophy	Measures of information; Occam's razor, induction

9.2 Applications

Domain	Examples
Everyday software	Spam filters, voice recognition, computer games, automatic number-plate recognition (petrol stations, toll roads), vehicle stability systems, bank loan decisions
Healthcare	Diagnostics; breast cancer / diabetes / Covid-19 prediction; tumours on mammograms; radiology & radiotherapy; assistive medical tech (brain tumours, fetal imaging, cardiac analysis), personalized treatment, drug discovery, clinical research
Agriculture	Crop management, field-conditions management
Education	Personalized learning
Transportation	Traffic regulation, automated driving, vehicle quality assurance; traffic prediction (Google/Bing Maps: average times + real-time vehicle locations)
Manufacturing	Smart maintenance, better product development, quality improvement, market adaptation
Others	Virtual personal assistants (Google Assistant, Alexa, Siri, Cortana); social-media personalization; online fraud detection (identity theft, fake accounts, man-in-the-middle; feed-forward NN for genuine vs fraud transactions); stock trading (LSTM to predict trends); automatic translation (sequence-to-sequence learning, CNN image recognition, OCR); credit-card fraud, fake news, sentiment analysis, duplicate-question detection, Android malware classification, food & calorie detection, telecom customer churn
Classic successes (Mitchell)	**SPHINX** — recognizing spoken words (phonemes; neural nets & HMMs adapt to speakers, vocabularies, microphones, noise); **ALVINN** — drove unassisted at 70 mph for 90 miles on public highways; **NASA sky survey** — decision trees classify celestial objects from 3 TB of Palomar images; **TD-Gammon** — learned backgammon by playing >1 million games against itself, competitive with the human world champion

9.3 Key terminology ★ Very likely

Term	Meaning	Fruit example
Model / hypothesis	A specific representation learned from data by applying an ML algorithm	The learned rule that maps features → fruit name
Feature	An individual measurable property of the data; a set of numeric features = **feature vector**, fed as input to the model. Choose informative, discriminating, independent features (feature extractor)	Colour, smell, taste, weight
Target (label)	The value to be predicted by the model	Apple, orange, banana
Training	Giving inputs (features) with expected outputs (labels) so that we get a model that maps new data to the trained categories	Show 1000 labelled fruits
Prediction	Feeding new inputs to the ready model to get a predicted output	New fruit → "apple"
Training set / test set	Data used to learn / held-out data used to evaluate on unseen examples	80% / 20% split
Instance, attribute	One example (row) / one feature (column)	One fruit / "colour"

More data > better model > higher accuracy. The right algorithm depends on the problem statement (e.g. predict future stock prices), size/quality/nature of data, and algorithm complexity. Note: ML algorithms are not types of ML.

9.4 Key tasks of machine learning ★ Very likely

Task	Type	What it does	Algorithms / examples
Classification	Supervised	Predict a discrete class (Yes/No, True/False, spam/not)	Decision trees, Naive Bayes, logistic regression, KNN, SVM, AdaBoost
Regression	Supervised	Predict a numeric value (stock price, product price from demand)	Linear, tree-based, multivariate regression
Clustering	Unsupervised	Group similar items without labels	k-means, hierarchical; search engines, optimal placement of base stations
Association	Unsupervised	Find patterns of items that occur together	Apriori, FP-Growth; market-basket analysis (Amazon/Flipkart)
Density estimation	Unsupervised	Find statistical values that describe the data	—
Dimensionality reduction	Unsupervised	Reduce number of features (visualization, compression)	PCA

SCENARIO → TECHNIQUE (WITH JUSTIFICATION) — ASKED AS SHORT QUESTIONS

- Filtering spam emails → **classification** (output is a category).
- Predicting stock market price; how much will it rain → **regression** (numeric output).
- Facebook recognizing friends in your album; optimal placement of base stations → **clustering** (group similar items).
- Amazon/Flipkart suggesting products; Netflix recommending movies from past choices → **association rule mining**.

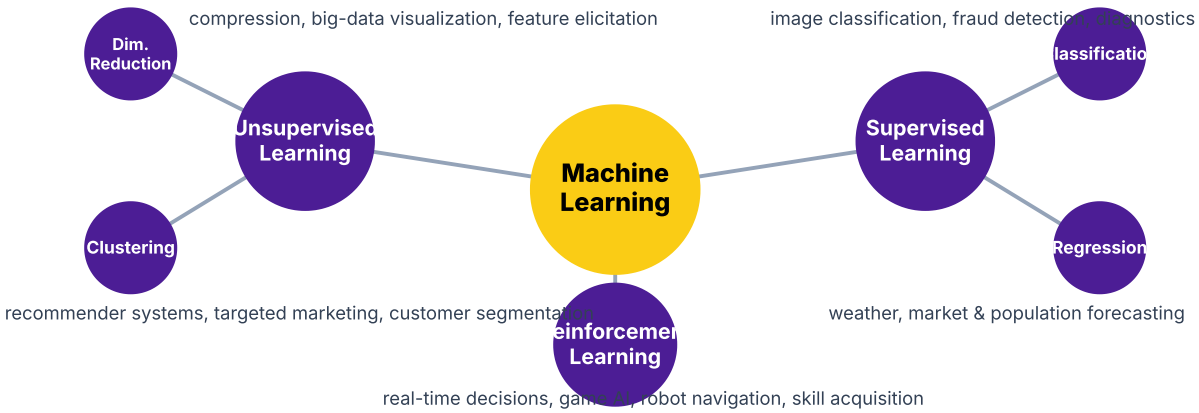


Fig 9.2 ML algorithms and where they are used.

9.5 Well-posed learning problems ★ Very likely

DEFINITION

A learning problem is **well-posed** (well-defined) when we identify three features: (1) the class of tasks T, (2) the measure of performance P to be improved, (3) the source of experience E.

Problem	Task T	Performance P	Experience E
Checkers	Playing checkers	% of games won against opponents	Playing practice games against itself
Handwriting recognition	Recognizing and classifying handwritten words within images	% of words correctly classified	A database of handwritten words with given classifications
Robot driving	Driving on public four-lane highways using vision sensors	Average distance travelled before an error (judged by human overseer)	Sequence of images and steering commands recorded while observing a human driver
Face recognition	Recognize faces	% of correct recognitions	Opportunity to make guesses and being told the truth
Finding clusters	Finding clusters in data	Compactness of groups detected	Analyses of a growing set of data
Traffic prediction	Predict traffic patterns at a busy intersection	Accuracy of future predictions	Data about past traffic patterns
Spam filtering	Classify emails as spam / not spam	% of emails correctly classified	Emails labelled by the user

CHECKERS — GAME BASICS (FROM THE SLIDES, IN CASE THEY ASK YOU TO DESCRIBE IT)

Two players, 12 discs each (black and red) on the 12 dark squares closest to them; board has 64 squares (32 dark, 32 light); black moves first, then alternate. Moves only on dark squares, so pieces move diagonally; single pieces move only forward; a non-capturing move goes one square; a capture (jump) leaps over one opponent piece diagonally; multiple jumps allowed in one turn; captures are compulsory (free choice if several). A piece reaching the farthest row is crowned king (a captured piece is placed on top); kings move diagonally forward and backward and can combine jumps in several directions. A player wins when the opponent cannot move (all pieces captured or blocked).

EXAM TIP

For "Explain well-posed learning problems", write Mitchell's definition, then the T/P/E table for at least **three** examples (checkers, handwriting, robot driving). If they give a new scenario, invent sensible T, P, E for it.

9.6 Types of machine learning ★ Very likely

How does the computer know whether it is getting better, and how does it know how to improve? The answer defines the type of learning:

Type	Idea	Examples / used for
Supervised	A training set of examples **with correct responses (targets)** is provided; the algorithm generalises to respond correctly to all possible inputs. Also called **learning from exemplars**.	Regression & classification (spam, price prediction)
Unsupervised	Correct responses are **not** provided; the algorithm identifies similarities between inputs so that inputs with something in common are categorized together. The statistical approach is called **density estimation**.	Clustering, PCA (customer segmentation)
Reinforcement	Between supervised and unsupervised: the algorithm is told **when the answer is wrong, but not how to correct it**; it must explore and try possibilities until it gets it right. Called **learning with a critic** (the monitor scores the answer but doesn't suggest improvements). Making decisions sequentially.	Game AI, robot navigation, real-time decisions
Evolutionary	Biological evolution as learning: organisms adapt to improve survival and offspring. Modelled using **fitness** — a score of how good the current solution is.	Genetic algorithms, optimization

MEMORY TRICK

Supervised = **S**tudent with a teacher (answers given). **U**nsupervised = **U**nguided explorer (finds groups). **R**einforcement = **R**eward/penalty from a critic. **E**volutionary = **E**volve the fittest.

9.7 Supervised learning in more depth

- **Classification:** inputs are divided into two or more classes; the learner produces a model that assigns unseen inputs to one or more classes (multi-label). Spam filtering: inputs = emails, classes = spam/not spam. The problem is **discrete** — each example belongs to precisely one class, and the set of classes covers the whole output space. These constraints aren't always realistic: examples may belong partially to two classes → **fuzzy classifiers**; some inputs may not belong to any trained class.
- **Regression:** outputs are **continuous** (e.g. stock prices from historical data). In statistics: fit a mathematical function (curve) passing as close as possible to all data points — **function approximation / interpolation** (working out values between known values); easy in 2-D, harder in high dimensions.

THE COIN CLASSIFIER (VENDING MACHINE)

- When a coin is pushed in, the machine measures features: **diameter, weight, shape** → a 3-element input vector (shape encoded as a number: 1 = circle, 2 = hexagon...). Could also use density, composition, or a photo.
- Trained on all New Zealand coins — what if a British coin is inserted? It will be labelled as the nearest NZ coin; really it should say "not one of my coins" → **novelty detection**.
- Which features to choose isn't easy: too many inputs make training slower, and as the number of input dimensions grows, the number of data points required increases even faster → curse of dimensionality.
- We need features that reliably separate classes, and **decision boundaries** that separate the classes.

9.8 The machine learning process ★ Likely

1. **Data collection and preparation** — collect from scratch or assemble and prepare; for a new problem merge with feature selection (collect a small dataset with all candidate features, experiment, then collect the full dataset). Supervised learning also needs **target data** (often requires

domain experts and time). Consider quantity: ML needs lots of data with little noise, but larger data = more computation; whether there is enough data is hard to predict.

2. **Feature selection** — identify the features most useful for the problem (e.g. coin recognition); requires prior knowledge; features should be cheap to collect and robust to noise.
3. **Algorithm choice** — needs knowledge of each algorithm's principles and examples of use.
4. **Parameter and model selection** — many algorithms have parameters set manually or by experimentation.
5. **Training** — use computational resources to build a model of the data that predicts outputs on new data.
6. **Evaluation** — test for accuracy on data it was **not trained on** before deployment; may compare with human experts using appropriate metrics.

CHALLENGES OF ML

- Data representation
- Needs massive stores of training data
- Labelling training data is tedious
- Machines cannot explain themselves
- Bias in the data

ADVANTAGES / LIMITATIONS

Advantages: no knowledge engineers needed (everything from data); scalable in constructing a knowledge base; adaptive to changing conditions; easy migration to new domains.
Limitations (classical ML): struggles with high-dimensional data; image/object recognition hard; features must be extracted manually (tedious with huge datasets); can't generate features on its own → **why deep learning**: works on high-dimensional data, learns the right features itself.

Frameworks: TensorFlow, Theano, Scikit-learn, Caffe, Amazon ML, Google Cloud ML Engine, Azure ML Studio, Spark MLlib.

9.9 Concept learning: Find-S & Candidate-Elimination ★ Very likely (numericals!)

CONCEPT LEARNING

Inferring a boolean-valued function (the **target concept** c: X → {0,1}) from training examples of its input and output. It can be viewed as searching through a large space of hypotheses H for the hypothesis that best fits the training examples.

The EnjoySport concept-learning task

- **Instances X:** all possible days, described by Sky (Sunny, Cloudy, Rainy), AirTemp (Warm, Cold), Humidity (Normal, High), Wind (Strong, Weak), Water (Warm, Cool), Forecast (Same, Change).
- **Target concept c:** EnjoySport: X → {0, 1}.
- **Hypotheses H:** each hypothesis is a **conjunction of constraints** on the attributes. Each constraint is a specific value (e.g. Warm), **"?"** (any value acceptable) or **"Ø"** (no value acceptable).
- **Training examples D:** positive and negative examples with c(x). **Determine** h ∈ H such that h(x) = c(x) for all x in D.
- Most general hypothesis: ⟨?, ?, ?, ?, ?, ?⟩ (every day is positive). Most specific: ⟨Ø, Ø, Ø, Ø, Ø, Ø⟩ (no day is positive).
- **Satisfies:** x satisfies h when h(x) = 1, regardless of whether x is positive or negative. **Consistent:** h is consistent with D when h(x) = c(x) for every example.
- **More-general-than-or-equal (≥g):** $h_j \geq_g h_k$ iff every instance satisfying h_k also satisfies h_j. E.g. h_2 = ⟨Sunny, ?, ?, ?, ?, ?⟩ is more general than h_1 = ⟨Sunny, ?, ?, Strong, ?, ?⟩. This partial order organizes the search.

Ex	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Find-S algorithm — find the maximally specific hypothesis

```
1. Initialize h to the most specific hypothesis in H:  h = <Ø, Ø, Ø, Ø, Ø, Ø>
2. For each POSITIVE training instance x:
      For each attribute constraint a_i in h:
          If x's value for a_i satisfies the constraint (equal) → do nothing
          Else → replace a_i in h by the next more general constraint ('?')
3. Output hypothesis h            (negative examples are IGNORED)
```

FIND-S ON ENJOYSPORT

$h_0 = \langle Ø, Ø, Ø, Ø, Ø, Ø \rangle$

x_1 (+): $h_1 = \langle$Sunny, Warm, Normal, Strong, Warm, Same$\rangle$ (copy the first positive example)

x_2 (+): Humidity differs (Normal vs High) → ? $h_2 = \langle$Sunny, Warm, ?, Strong, Warm, Same$\rangle$

x_3 (−): ignored → $h_3 = h_2$

x_4 (+): Water (Warm vs Cool) and Forecast (Same vs Change) differ → h_4 =

⟨Sunny, Warm, ?, Strong, ?, ?⟩

FIND-S — POISONOUS SEEDS (CLASS)

#	Color	Toughness	Fungus	Appearance	Poiso
1	Green	Hard	No	Wrinkled	Ye
2	Green	Hard	Yes	Smooth	No
3	Brown	Soft	No	Wrinkled	No
4	Orange	Hard	No	Wrinkled	Ye
5	Green	Soft	Yes	Smooth	Ye
6	Green	Hard	Yes	Wrinkled	Ye
7	Orange	Hard	No	Wrinkled	Yes

$h_0 = \langle Ø, Ø, Ø, Ø \rangle$

x_1+: $h_1 = \langle$Green, Hard, No, Wrinkled$\rangle$

x_2−, x_3−: ignore → $h_3 = h_1$

x_4+ (Orange): $h_4 = \langle$?, Hard, No, Wrinkled$\rangle$

x_5+ (Green, Soft, Yes, Smooth): $h_5 = \langle$?, ?, ?, ?$\rangle$

x_6+, x_7+: no change

Final: **⟨?, ?, ?, ?⟩**

FIND-S — GOING FOR A WALK

Time	Weather	Temp	Company	Humidity	Wind	Goes
Morning	Sunny	Warm	Yes	Mild	Strong	Yes
Evening	Rainy	Cold	No	Mild	Normal	No
Morning	Sunny	Moderate	Yes	Normal	Normal	Yes
Evening	Sunny	Cold	Yes	High	Strong	Yes

$h_1 = \langle$Morning, Sunny, Warm, Yes, Mild, Strong$\rangle$ → $h_3 = \langle$Morning, Sunny, ?, Yes, ?, ?$\rangle$ → h_4 =

⟨?, Sunny, ?, Yes, ?, ?⟩

FIND-S — CAR DATASET (JAPANESE ECONOMY CAR)

Org	Manf	Color	Year	Type	Class
JP	HO	Blue	1980	ECO	+
JP	TO	Green	1970	SPO	−
JP	TO	Blue	1990	ECO	+
USA	CH	Red	1980	ECO	−
JP	HO	White	1980	ECO	+
JP	TO	Green	1980	ECO	+
JP	HO	Red	1990	ECO	−

h_1 = ⟨JP, HO, Blue, 1980, ECO⟩ → h_3 = ⟨JP, ?, Blue, ?, ECO⟩ → h_5 = h_6 = **⟨JP, ?, ?, ?, ECO⟩**. (Note: this h wrongly covers negative example 7 — Find-S can't detect that because it ignores negatives.)

PROPERTIES & SHORTCOMINGS OF FIND-S

- Guaranteed to output the **most specific** hypothesis in H consistent with the positive examples (if the correct target is in H and data has no errors).
- Can't tell whether it has learned the correct concept (no way to know if it's the only consistent one). Is preferring the most specific hypothesis a good strategy?
- Can't detect inconsistent (noisy) training data. If there are several maximally specific consistent hypotheses, it can't backtrack.
- **Negative examples are not considered.**

Version space

DEFINITION

The **version space** $VS_{H,D}$ is the subset of hypotheses from H consistent with all training examples in D: $VS_{H,D}$ = { h ∈ H | Consistent(h, D) }. It can be represented by its **general boundary G** (maximally general consistent hypotheses) and **specific boundary S** (maximally specific consistent hypotheses).

LIST-THEN-ELIMINATE algorithm: (1) VersionSpace ← a list of every hypothesis in H; (2) for each training example ⟨x, c(x)⟩ remove any h with h(x) ≠ c(x); (3) output the list. It works whenever H is finite and is guaranteed to output all consistent hypotheses, but requires exhaustively enumerating H — unrealistic for all but trivial spaces.

Candidate-Elimination algorithm ★ Very likely

Finds **all** describable hypotheses consistent with the observed examples, using **both positive and negative** examples, by maintaining S and G.

```
Initialize G ← maximally general hypotheses in H   (<?,?,…,?>)
Initialize S ← maximally specific hypotheses in H  (<Ø,Ø,…,Ø>)
For each training example d:
 • If d is POSITIVE:
     – Remove from G any hypothesis inconsistent with d
     – For each s in S not consistent with d: remove s, and add to S all minimal
       generalizations h of s such that h is consistent with d and some member of G
       is more general than h
     – Remove from S any hypothesis more general than another hypothesis in S
 • If d is NEGATIVE:
     – Remove from S any hypothesis inconsistent with d
     – For each g in G not consistent with d: remove g, and add to G all minimal
       specializations h of g such that h is consistent with d and some member of S
       is more specific than h
     – Remove from G any hypothesis less general than another hypothesis in G
```

MEMORY TRICK

Positive → S grows (generalize), G gets pruned. Negative → G shrinks (specialize), S gets pruned.
"S moves up on Yes, G moves down on No."

CANDIDATE-ELIMINATION ON ENJOYSPORT (STEP BY STEP)

Init: $G_0 = \{\langle ?, ?, ?, ?, ?, ?\rangle\}$, $S_0 = \{\langle Ø, Ø, Ø, Ø, Ø, Ø\rangle\}$

x_1 = ⟨Sunny, Warm, Normal, Strong, Warm, Same⟩ (+): $S_1 = \{\langle$Sunny, Warm, Normal, Strong, Warm, Same$\rangle\}$, $G_1 = G_0$

x_2 = ⟨Sunny, Warm, High, Strong, Warm, Same⟩ (+): $S_2 = \{\langle$Sunny, Warm, ?, Strong, Warm, Same$\rangle\}$, $G_2 = G_0$

x_3 = ⟨Rainy, Cold, High, Strong, Warm, Change⟩ (−): $S_3 = S_2$. Specialize G minimally so x_3 is excluded but each member stays more general than S_2:

$G_3 = \{\langle$Sunny, ?, ?, ?, ?, ?$\rangle, \langle$?, Warm, ?, ?, ?, ?$\rangle, \langle$?, ?, ?, ?, ?, Same$\rangle\}$ (⟨?,?,Normal,…⟩ and ⟨?,?,?,?,Cool,?⟩ rejected — not more general than S_2)

x_4 = ⟨Sunny, Warm, High, Strong, Cool, Change⟩ (+): $S_4 = \{\langle$Sunny, Warm, ?, Strong, ?, ?$\rangle\}$; remove ⟨?, ?, ?, ?, ?, Same⟩ from G (inconsistent with x_4)

$G_4 = \{\langle$Sunny, ?, ?, ?, ?, ?$\rangle, \langle$?, Warm, ?, ?, ?, ?$\rangle\}$

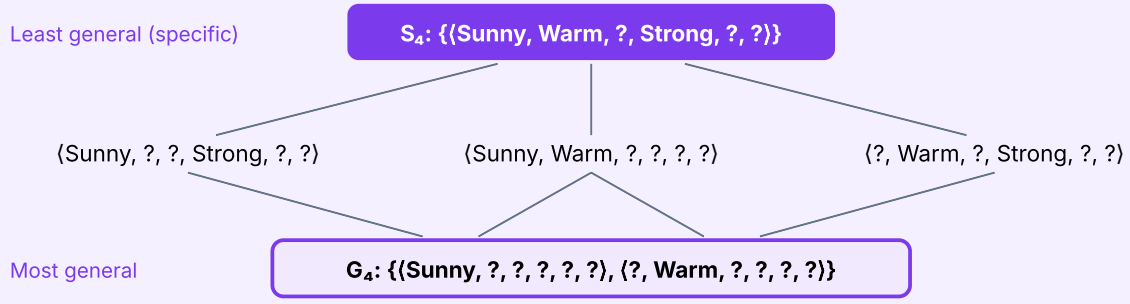


Fig 9.3 Final version space for EnjoySport: every hypothesis between S_4 and G_4 is consistent with all 4 examples (6 hypotheses in total).

CANDIDATE-ELIMINATION — "SMALL CIRCLES" (CLASS)

Size	Color	Shape	Class
big	red	circle	No
small	red	triangle	No
small	red	circle	Yes
big	blue	circle	No
small	blue	circle	Yes

$G_0 = \{(?, ?, ?)\}$, $S_0 = \{(\emptyset, \emptyset, \emptyset)\}$

1 (big, red, circle) −: minimal specializations: $G_1 = \{\langle small, ?, ?\rangle, \langle ?, blue, ?\rangle, \langle ?, ?, triangle\rangle\}$; $S_1 = S_0$

2 (small, red, triangle) −: ⟨small,?,?⟩ covers it → ⟨small, blue, ?⟩, ⟨small, ?, circle⟩; ⟨?,blue,?⟩ is fine; ⟨?,?,triangle⟩ covers it → ⟨big, ?, triangle⟩, ⟨?, blue, triangle⟩

$G_2 = \{\langle small, blue, ?\rangle, \langle small, ?, circle\rangle, \langle ?, blue, ?\rangle, \langle big, ?, triangle\rangle, \langle ?, blue, triangle\rangle\}$; $S_2 = S_0$ (as on the slide)

Strictly, the last step of the algorithm removes ⟨small, blue, ?⟩ and ⟨?, blue, triangle⟩ because they are less general than ⟨?, blue, ?⟩ → $G_2 = \{\langle small, ?, circle\rangle, \langle ?, blue, ?\rangle, \langle big, ?, triangle\rangle\}$. The final answer is the same either way.

3 (small, red, circle) +: $S_3 = \{\langle small, red, circle\rangle\}$; drop every g not covering it → $G_3 = \{\langle small, ?, circle\rangle\}$

4 (big, blue, circle) −: already excluded by G_3 and S_3 → no change

5 (small, blue, circle) +: $S_5 = \{\langle small, ?, circle\rangle\}$; $G_5 = \{\langle small, ?, circle\rangle\}$

S = G ⇒ converged: the learned concept is **"small circles" ⟨small, ?, circle⟩**

CANDIDATE-ELIMINATION — JAPANESE ECONOMY CAR (VERSION SPACE COLLAPSES)

$S_0 = \langle \Phi, \Phi, \Phi, \Phi, \Phi\rangle$, $G_0 = \langle ?, ?, ?, ?, ?\rangle$

x_1 ⟨Japan, Honda, Blue, 1980, Economy⟩ +: $S_1 = \langle$Japan, Honda, Blue, 1980, Economy$\rangle$; $G_1 = G_0$

x_2 ⟨Japan, Toyota, Green, 1970, Sports⟩ −: $S_2 = S_1$; $G_2 = \{\langle ?, Honda, ?, ?, ?\rangle, \langle ?, ?, Blue, ?, ?\rangle, \langle ?, ?, ?, 1980, ?\rangle, \langle ?, ?, ?, ?, Economy\rangle\}$

x_3 ⟨Japan, Toyota, Blue, 1990, Economy⟩ +: $S_3 = \langle$Japan, ?, Blue, ?, Economy$\rangle$; $G_3 = \{\langle ?, ?, Blue, ?, ?\rangle, \langle ?, ?, ?, ?, Economy\rangle\}$

x_4 ⟨USA, Chrysler, Red, 1980, Economy⟩ −: $S_4 = S_3$; $G_4 = \{\langle ?, ?, Blue, ?, ?\rangle, \langle$Japan, ?, ?, ?, Economy$\rangle\}$

x_5 ⟨Japan, Honda, White, 1980, Economy⟩ +: $S_5 = \langle$Japan, ?, ?, ?, Economy$\rangle$; $G_5 = \{\langle$Japan, ?, ?, ?, Economy$\rangle\}$ (Blue removed)

x_6 ⟨Japan, Toyota, Green, 1980, Economy⟩ +: $S_6 = G_6 = \langle$Japan, ?, ?, ?, Economy$\rangle$

x_7 ⟨Japan, Honda, Red, 1990, Economy⟩ −: S covers this negative → S must be removed; G can't be specialized while staying above S → **the version space collapses** (S and G become empty).

Conclusion: **no conjunctive hypothesis is consistent with the data set** (example 7 is inconsistent with the version space).

REMARKS ON VERSION SPACES

The version space learned by Candidate-Elimination converges toward the hypothesis that correctly describes the target concept, provided (1) there are no errors in the training examples and (2) some hypothesis in H correctly describes the target concept. When S and G become identical, the concept is learned exactly; if they become empty, the data is noisy or the target isn't in H.

Algorithm	Order / direction	Strategy	Examples used
Find-S	Specific-to-general	Top-down	Positive only
List-Then-Eliminate	General-to-specific	Bottom-up	Negative (and positive) — eliminates
Candidate-Elimination	Bi-directional	Bi-directional	Both

(Table reproduced as given on the class slide.)

LIKELY EXAM QUESTIONS — CH 9

1. Define machine learning (Samuel & Mitchell). Differentiate traditional programming and ML with a diagram. (5M)
2. Explain key terminology: model, feature, target, training, prediction. (5M)
3. What are the key tasks of ML? For each given scenario, which type of learning is used — justify. (5M)
4. What is a well-posed learning problem? Give T, P, E for checkers / handwriting / robot driving. (6M) ★
5. Explain the types of ML with examples. (6M) ★
6. Explain the steps in the ML process. (5M)
7. Apply Find-S to the given dataset and find the maximally specific hypothesis. (6–8M) ★
8. Apply Candidate-Elimination; show S and G after each example. Define version space. (10M) ★

60-SECOND RECAP

Mitchell: learn from E w.r.t. T and P. Terms: model, feature, label, training, prediction. Tasks: classification, regression (supervised); clustering, association, density estimation, PCA (unsupervised). Types: supervised (exemplars), unsupervised (similarity), reinforcement (critic), evolutionary (fitness). Find-S: start Ø, generalize on positives only. Candidate-Elimination: S up on +, G down on −; version space = all consistent h between S and G.